ECE560

Winter 2025

2 Wheel Balancing Robot

Team:
Folarin Davies
Scott Cann

Table of Contents

1. Project Summary
2. System Modelling and State Space Development
3. Summarized Motion Equations and State Space Representation
4. Sensed and Observed States
5. Controllability and Observability
6. Initial Controller Design
7. LQR Controller Development
8. Observer Design
9. Robustness Testing
10. Conclusion
11. References/Sources
12. Appendix

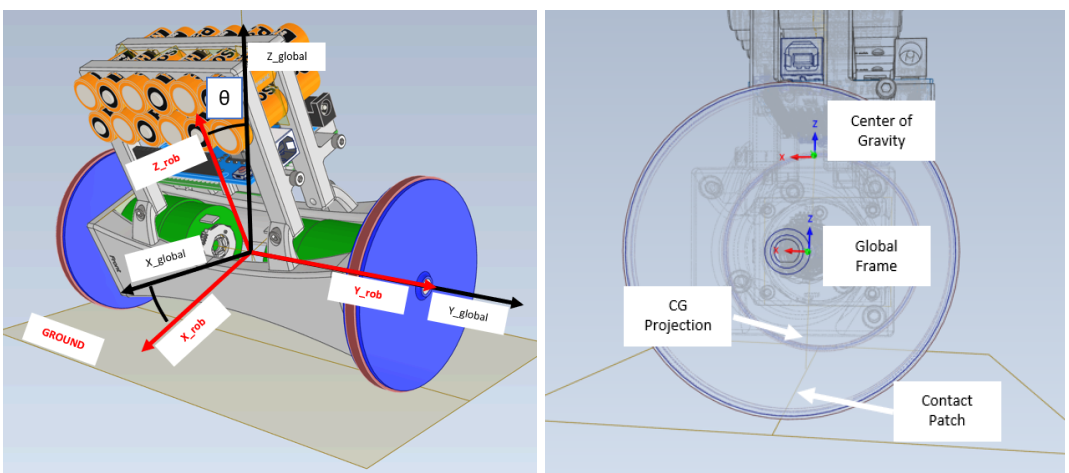
ECE 560 Winter 2025
Final Project Report
Scott Cann, Folarin Davies

Project Summary

In this paper, we consider a two wheel balancing robot controlled by independent DC motors. Control dynamics (both electrical and mechanical) are considered, and 6th order state space models are developed based on parameters derived from a detailed CAD model and real-world measurements. A controller and Kalman filter observer were designed with full state feedback, then tested in Simulink for robustness against sensor noise and parameter variations. In the end we were able to successfully control balance, forward velocity, and steering velocity of this inherently unstable system.

System Modelling and State Space Development:

A two wheel balancing robot was modelled in CAD with material properties to determine nominal values for inertia, mass and other critical parameters described in *Appendix 1*. DC motors were used to drive the left and right wheels for mobility and balance.



The robot coordinate frame (red) has its Y-axis permanently collinear with the drive wheel axis, and Z-axis pointing up when the robot's center of gravity projects onto the wheel contact patch line (unstable equilibrium position, figure 2). Our primary goal is to maintain stability ($\theta \rightarrow 0°$ from initial condition of $\theta \leq 10°$) while controlling motion in the global frame, as expressed by the robots forward velocity $\dot{x}$ and steering velocity, ϕ_{dot}. $\dot{x}$ here is the projection of $\dot{x}_{rob}$ onto the XY_{global} frame, and ϕ_{dot} is the rate of angle change between Y_{rob} and Y_{global}. A no-slip condition is a reasonable approximation which leads to a direct relationship between wheel speeds and robot velocities:

1. $\dot{x} = \frac{w_{right}r + w_{left}r}{2}$
2. $\phi_{dot} = \frac{w_{right}r - w_{left}r}{trackwidth}$
3. $\frac{d}{dt}(w_{left}) = \frac{1}{2r}(2\ddot{x} - \frac{d}{dt}(\phi_{dot})trackwidth)$
4. $\frac{d}{dt}(w_{right}) = \frac{1}{2r}(2\ddot{x} + \frac{d}{dt}(\phi_{dot})trackwidth)$

Left and right wheel velocities (w_{left}, w_{right}) become obvious state variable choices, as they describe motion in the global frame. Motor control is achieved by pulse width modulated voltage to the left and right motor circuits. v_{left}, v_{right} become our system inputs. I_{left}, I_{right} are two system states, since they directly relate to motor torque by $T = K_t I$, which drives wheel velocities. From motor specification *Appendix 1*, we derive $K_t = 980Nm/A$. Direct multi-meter measurement yielded motor parameters ($R = 3\Omega,\ L = 150mH$). Input voltages will saturate at 4V so that steady state motor current cannot exceed $I = \frac{4V}{3\Omega} \simeq 1.3A$, which is a safe limit. A sample scheme for applying variable voltage for a single motor direction is given below. The governing *motor circuit* equations are also given, with their full derivation in *Appendix 2*.

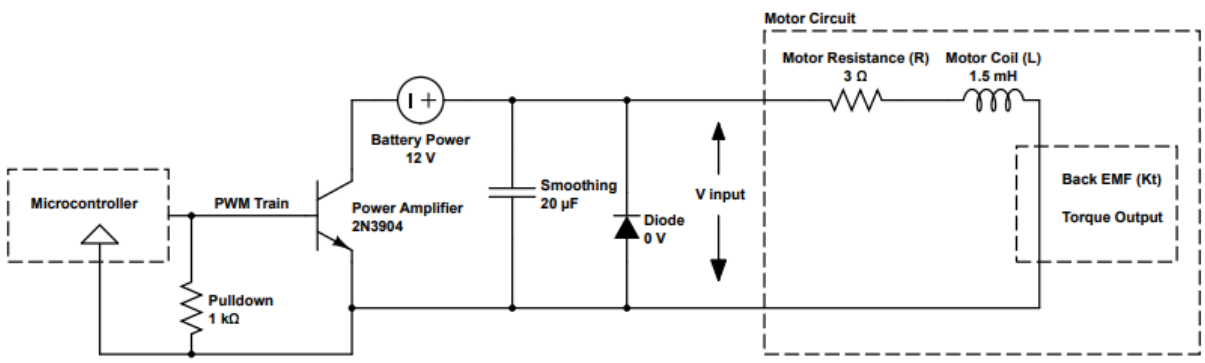


Figure1: Motor circuit example for controlling input voltage

5. $\frac{d}{dt}(I_{left}) = \frac{-R}{L}I_{left} - \frac{K_t}{L}w_{left} + \frac{1}{L}v_{left}$
6. $\frac{d}{dt}(I_{right}) = \frac{-R}{L}I_{right} - \frac{K_t}{L}w_{right} + \frac{1}{L}v_{right}$

Free body diagrams of the robot body and robot wheels were used to derive the balance (θ) and driving ($\dot{x}$) dynamics, where T_m is expressed directly in terms of motor currents by $T_m = T_{right} + T_{left} = K_t I_{right} + K_t I_{left}$. Since we are concerned primarily with low speed (ω_L, ω_R) balancing, rolling resistance and mechanical (motor) damping is considered insignificant, while inertias dominate. Small angle

approximations were made to linearize Eq.5, since only small perturbations from the unstable equilibrium ($\theta = 0$) will be controlled. See *Appendix 2*.

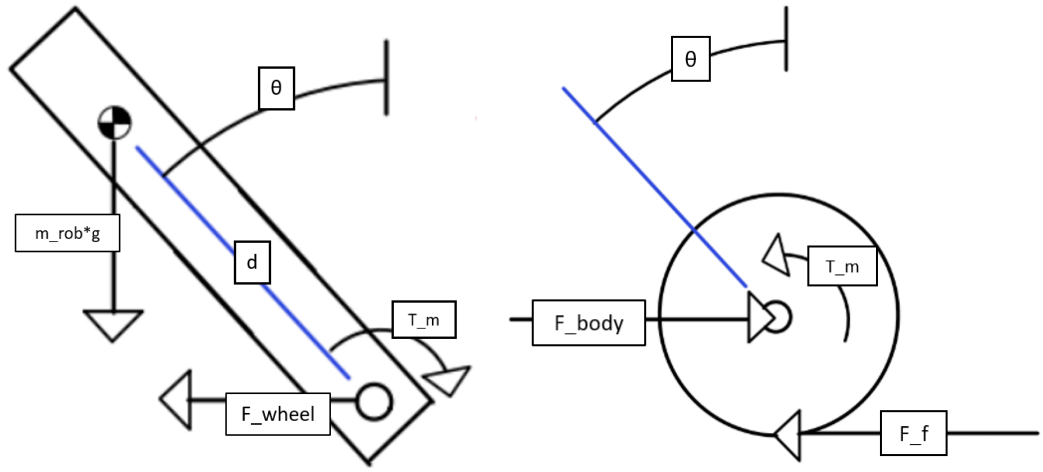


Figure2: FBD for wheel and body dynamics

7. $$\ddot{x} = \frac{K_t r(I_{right} + I_{left})}{r^2 m_{rob} + r^2 m_{wheel} + J_{wheel}}$$

8. $$\frac{d}{dt}(\theta) = \dot{\theta}$$

9. $$\frac{d}{dt}(\dot{\theta}) = \frac{m_{rob} g d}{I_{yy}}\theta + \left(\frac{m_{rob} d K_t r}{I_{yy}(r^2 m_{rob} + r^2 m_{wheel} + J_{wheel})} + \frac{K_t}{I_{yy}}\right) I_{left} + \left(\frac{m_{rob} d K_t r}{I_{yy}(r^2 m_{rob} + r^2 m_{wheel} + J_{wheel})} - \frac{K_t}{I_{yy}}\right) I_{right}$$

A top view of the robot is taken to derive steering (ϕ_{dot}) dynamics.

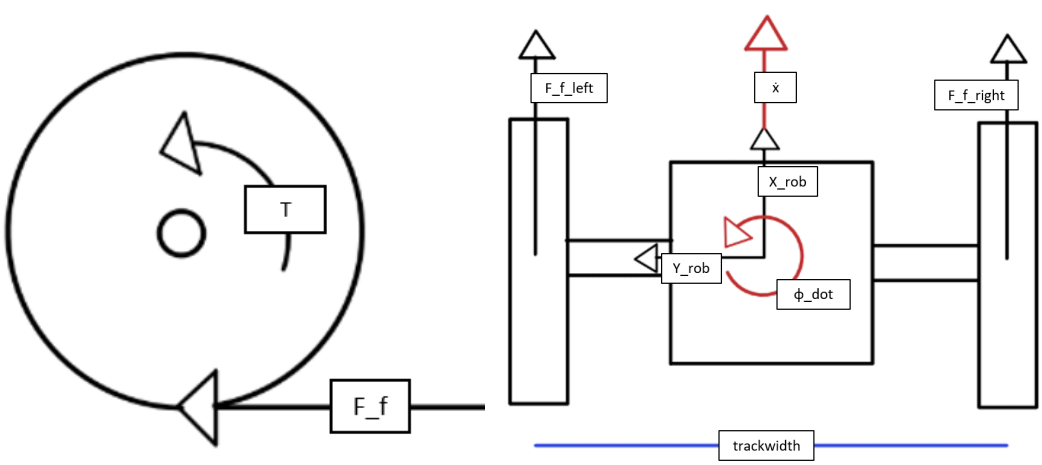


Figure3: FBD for Yaw Dynamics

Torque differential between left and right wheels causes yaw (ϕ) acceleration of the robot, against the resistance of wheel inertia (J_{wheel}) and robot inertia (I_{zz}). $I_{zz} = f(\theta)$, which we can approximate as $I_{zz} = I_{zz}(\theta = 0)$ since perturbations from the unstable equilibrium are minimal, and steering (ϕ) control is less critical than balance (θ) control.

10. $\frac{d}{dt}(\phi_{dot}) = \frac{(K_t)trackwidth}{2r(I_{zz} + \frac{trackwidth}{r}J_{wheel})} I_{right} - \frac{(K_t)trackwidth}{2r(I_{zz} + \frac{trackwidth}{r}J_{wheel})} I_{left}$

11. $\frac{d}{dt}(\phi) = \phi_{dot}$

All nominal model parameter values and their ranges are given in **(appendix 1)**. Nominal values were used in the 6 motion equations to form the final state space representation, matrices A, B, C.

Summarized Motion Equations and State Space Representation

1. $\frac{d}{dt}(\theta) = \dot{\theta}$
2. $\frac{d}{dt}(\dot{\theta}) = \frac{m_{rob}gd}{I_{yy}}\theta + \left(\frac{m_{rob}dK_t r}{I_{yy}(r^2 m_{rob} + r^2 m_{wheel} + J_{wheel})} + \frac{K_t}{I_{yy}}\right) I_{left} + \left(\frac{m_{rob}dK_t r}{I_{yy}(r^2 m_{rob} + r^2 m_{wheel} + J_{wheel})} - \frac{K_t}{I_{yy}}\right) I_{right}$
3. $\frac{d}{dt}(I_{left}) = \frac{-R}{L} I_{left} - \frac{K_t}{L} w_{left} + \frac{1}{L} v_{left}$
4. $\frac{d}{dt}(I_{right}) = \frac{-R}{L} I_{right} - \frac{K_t}{L} w_{right} + \frac{1}{L} v_{right}$
5. $\frac{d}{dt}(w_{left}) = \frac{1}{2r}\left(2\ddot{x} - \frac{d}{dt}(\phi_{dot})trackwidth\right)$
6. $\frac{d}{dt}(w_{right}) = \frac{1}{2r}\left(2\ddot{x} + \frac{d}{dt}(\phi_{dot})trackwidth\right))$

Where:

$$\ddot{x} = \frac{K_t r(I_{right} + I_{left})}{r^2 m_{rob} + r^2 m_{wheel} + J_{wheel}}$$

$$\frac{d}{dt}(\phi_{dot}) = \frac{(K_t)trackwidth}{2r(I_{zz} + \frac{trackwidth}{r}J_{wheel})} I_{right} - \frac{(K_t)trackwidth}{2r(I_{zz} + \frac{trackwidth}{r}J_{wheel})} I_{left}$$

$$u = [v_{left};\ v_{right}]$$

$$\dot{\mathbf{x}} = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ -117.1 & 0 & 587.9 & -322.2 & 0 & 0 \\ 0 & 0 & -200 & 0 & -65.33 & 0 \\ 0 & 0 & 0 & -200 & 0 & -65.33 \\ 0 & 0 & 1959.1 & -1736.5 & 0 & 0 \\ 0 & 0 & -1736.5 & 1959.1 & 0 & 0 \end{bmatrix} \mathbf{x} + \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 66.67 & 0 \\ 0 & 66.67 \\ 0 & 0 \\ 0 & 0 \end{bmatrix} \mathbf{u}$$

$$\mathbf{x} = \begin{bmatrix} \theta \\ \dot{\theta} \\ I_{\text{left}} \\ I_{\text{right}} \\ \omega_{\text{left}} \\ \omega_{\text{right}} \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \mathbf{x}$$

Figure4: State Space Representation

Sensed and Observed States

The first two states (θ, $\dot{\theta}$) are directly measurable via a chassis mounted 6-axis IMU, which combines accelerometers, gyroscopes and magnetometers to sense angular position and velocity. Wheel speed (w_{left}, w_{right}) is sensed directly by a shaft mounted analogue tachometer. Current (I_{left}, I_{right}) could be sensed by a voltage reading across a sense resistor, but would wastefully dissipate battery power and add sensing overhead to the microcontroller. It is better to instead develop an observer for motor current from the other 4 sensed (output) states. System outputs become θ, $\dot{\theta}$, w_{left}, w_{right}. System inputs v_{left}, v_{right} are adjustable by PWM as discussed earlier, and I_{left}, I_{right} are observed.

Controllability and Observability

The controllability matrix formed from the A, B system matrices have rank of 6 (full rank), meaning all states are reachable using voltage inputs to the left and right motor circuits *Appendix 4*. The observability matrix formed from A, C were also full rank, meaning I_{left}, I_{right} are able to be observed given the 4 other sensed outputs *Appendix 4*. Open loop eigenvalues of the A matrix showed stability in I and w (negative real parts), and undamped oscillatory response (zero real part) for θ and $\dot{\theta}$, which is reasonable recalling that motor damping was omitted from the dynamic model *Appendix 2*.

Initial Controller Design

The MATLAB place command was used for initial system testing and simulation in simulink using full state feedback and *no observer* (C was set to a 6x6 identity matrix for full state output). A (2x4) K_{ref} matrix was developed so that the desired steady state outputs for w_{left}, w_{right}, θ, $\dot{\theta}$ could be fed (through it) into the system *Appendix 7.* Equations 3, 4 were used in the *WheelSpeeds* block so that wheel speed velocities could be set from the desired forward and turning velocities ($\dot{x}$, ϕ_{dot}) *Appendix 5*.

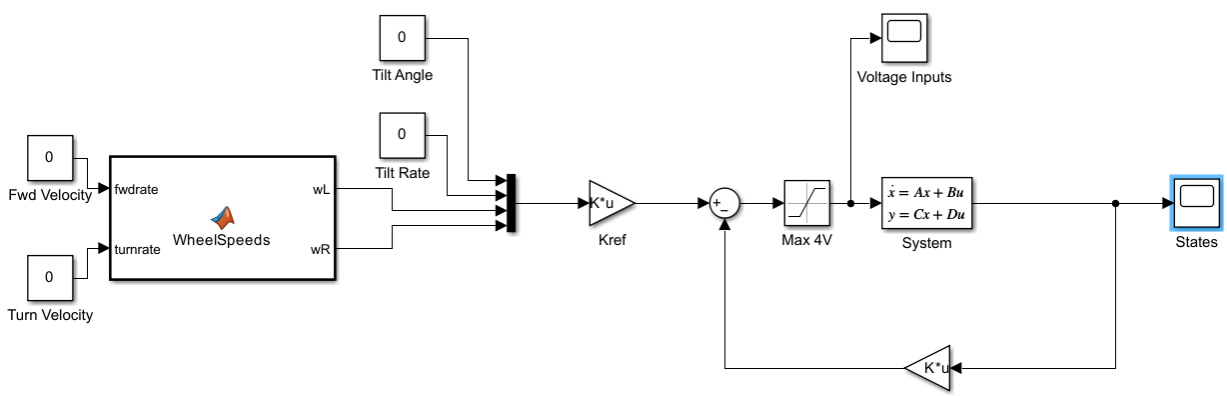


Figure5: Controller Design Simulation

Different controller feedback (*K*) matrices were formed using pole placement in MATLAB and evaluated by two performance metrics (approximate settling time for tilt angle $T_{S,\theta}$ and wheel speeds $T_{S,w}$) and two control effort metrics (Maximum current I and wheel speed w), starting from an initial condition of $\theta = 10°$. The feedback matrices generated by the place command were adjusted to ensure feedback gain for left and right wheel inputs (v_{left}, v_{right}) were matched, so that wheel speeds would stay synced unless a turn velocity was requested**.** Input saturation was set to 4V as discussed in the *System Modelling* section and *Appendix 1*. Results for 5 different pole-placed controllers are shown in *Appendix 8***.** Controller *K3* showed the best performance and settled in ~0.1s at the expense of large feedback gain and high control effort ($> 1A$ peak), compared to *K5* which settled in ~0.35s with significantly lower control effort ($< 0.5A$ peak). An optimized LQR controller was developed next to see if performance could be improved further from the *K5* controller with similarly low control effort.

LQR Controller Development

The state cost matrix (Q_{LQR}) and control cost matrix (R_{LQR}) were defined according to

$$\boldsymbol{Q} = \begin{pmatrix} q_1 & 0 & \cdots & \cdots & 0 \\ 0 & q_2 & 0 & \cdots & 0 \\ \vdots & \ddots & \vdots & \ddots & 0 \\ 0 & 0 & \cdots & 0 & q_n \end{pmatrix} \quad with \quad q_i = \frac{1}{t_{si} x_{i\,max}^2} \qquad \boldsymbol{R} = \rho \begin{pmatrix} r_1 & 0 & \cdots & \cdots & 0 \\ 0 & r_2 & 0 & \cdots & 0 \\ \vdots & \ddots & \vdots & \ddots & 0 \\ 0 & 0 & \cdots & 0 & r_n \end{pmatrix} with \quad r_i = \frac{1}{u_{i\,max}^2}$$

where our cost function is

$$J \equiv \int_t^{\infty} \left[\boldsymbol{x}^T(\boldsymbol{\tau}) \boldsymbol{Q} \boldsymbol{x}(\boldsymbol{\tau}) + \boldsymbol{u}^T(\boldsymbol{\tau}) \boldsymbol{R} \boldsymbol{u}(\boldsymbol{\tau}) \right] d\tau$$

t_{si} is the desired settling time of state i, with max value x_i, and u_i is the max value of input i. Settling times and maximum state values from the *K3* (higher performance, higher control effort) and *K5* (lower performance, lower control effort) controlled systems were used to arrive at target values for the LQR controller, as shown in the table below.

Parameter	Assigned Value	Reason/Source
$t_{s\theta}$	0.1	Average $t_{s\theta}$ from *K3* and *K4* controlled systems, which both sufficiently corrected $\theta = 10°$
$t_{s\theta_{dot}}$	0.1	$t_{s\theta_{dot}}$ from the faster *K3* controller
$t_{sI_{left}}$	0.25	Approximate $t_{sI_{left}}$ for *K1*, *K2*, *K3*, and *K4* controlled systems. Less critical than $t_{s\theta}$ and $t_{s\theta_{dot}}$
$t_{sI_{right}}$	0.25	Same as above
$t_{sw_{left}}$	0.25	Approximate $t_{sw_{left}}$ for K1, K2, K3, and K4 controlled systems. Less critical than $t_{s\theta}$ and $t_{s\theta_{dot}}$
$t_{sw_{left}}$	0.25	Same as above
$u_1\ (v_{left})$	4	Maximum 4V input allowed (discussed in modelling section)
$u_1\ (v_{right})$	4	Maximum 4V input allowed (discussed in modelling section)

x_{θ}	1	This is far from balanced region, should never exceed this
$x_{\theta_{dot}}$	5	None of the *K1-5* controlled systems needed to exceed this value to balance from $\theta = 10°$
$x_{I_{left}}$	1	Current to motor should not exceed 1A, discussed in modelling section
$x_{I_{right}}$	1	Same as above
$x_{w_{left}}$	4	*K1-5* controlled systems peak at around this value, with *K4* at $3.9\frac{rad}{s}$
$x_{w_{left}}$	4	Same as above

Figure6: LQR Design Table

MATLAB's icare() function was used to solve the algebraic riccati equation given the Q_{LQR} and $\varphi \cdot R_{LQR}$ matrices, and generate the optimal feedback gain matrix K_{LQR}, where φ is a scalar which adjusts the cost weighting of control effort. System response to a $\theta = 10°$ initial condition with K_{LQR} for $\varphi = 10,\ \varphi = 1,\ \varphi = 0.1,\ \varphi = 0.01$ were simulated, see *Appendix 10*. The K_{LQR} for $\varphi = 10$ gave somewhat similar performance to controller *K5*. However, *K5* contained identical gains for left and right feedback, which made the robot stabilize robot tilt without differential wheel speeds (ϕ_{dot}=0). Settling time $t_{s\theta}$ was also better, with similar control effort. For these reasons, *K5* was chosen as the final controller design.

Observer Design

Since our system is fully observable, all 6 observer poles are able to be placed arbitrarily. To help the observer rapidly minimize tracking error between the observed and the true system states, the MATLAB place command was used to place the observer poles 25 times further into the left hand plane than the poles of the chosen controller, *K4 appendix 8*. This led to good performance and low state estimation error under normal conditions. However, stability was quickly lost when sensor noise was introduced to the system. A Kalman filter observer was implemented (*Appendix 11)* in its place and simulated in the final system, which placed more weight on model prediction than sensor feedback (assumes higher sensor noise than model noise). As shown below, the $[v_{left};\ v_{right}]$ input vector is fed into the system and observer, which both

output a 6 dimension state vector. Both outputs are multiplied by the system C matrix before being compared, so that error between the observer prediction and *sensed* output variables ($\theta, \dot{\theta}, w_{left}, w_{right}$) can be fed back into the observer, and used to minimize tracking error for the unmeasured states (I_{left}, I_{right}).

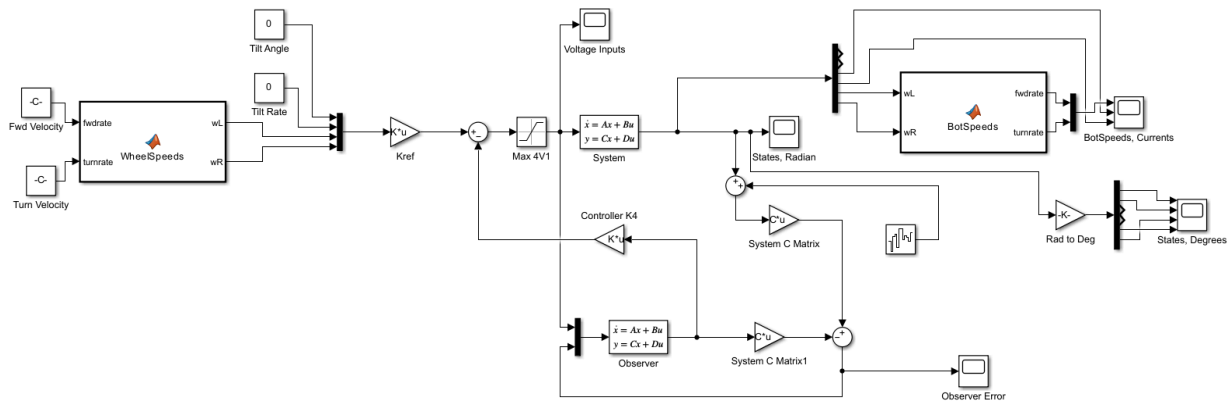


Figure7: Simulink Simulation Model

Robustness Testing

To test the performance and robustness of the controller/observer system for maintaining stability while reaching target velocities ($\dot{x}$, ϕ_{dot}), gaussian zero-mean white noise with a 1° (0.0349 radian) standard deviation was injected into all 4 sensed output states. Based on the below test matrix, model parameter values were varied between their low (-1), nominal (0), and high (1) values from *Appendix 1*. The last three columns indicate the initial condition of θ ($0 \rightarrow 0°$, $1 \rightarrow 10°$), target forward velocity $\dot{x}$ ($0 \rightarrow 0\frac{m}{s}$, $1 \rightarrow 0.1\frac{m}{s}$), and target turning velocity ϕ_{dot}($0 \rightarrow 0\frac{rad}{s}$, $1 \rightarrow 0.1\frac{rad}{s}$). In this manner, the nominal, upper limit, and lower limit “systems” and two randomly generated combinations are tested in the presence of signal noise for stability. (These parameter variations affect the system matrix only, to test if the control scheme is robust against error in the modelling. The observer remains defined by the nominal values, since it is part of the design we are testing). The test matrix MATLAB implementation is in *Appendix 12*, full results are plotted in *Appendix 13*.

Test #	I_{yy}	I_{zz}	r	m_{rob}	m_{wheel}	J_{wheel}	$track\ width$	d	g	Kt	L	R	$\theta_{initial}$	$\dot{x}_{target}$	$\phi_{dot\ target}$
1	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0

2	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0
3	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
4	1	1	1	1	1	1	1	1	1	1	1	1	1	0	0
5	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	1	0	0
6	-1	0	-1	0	1	-1	1	-1	-1	0	1	1	1	0	1

Figure8: Test Matrix

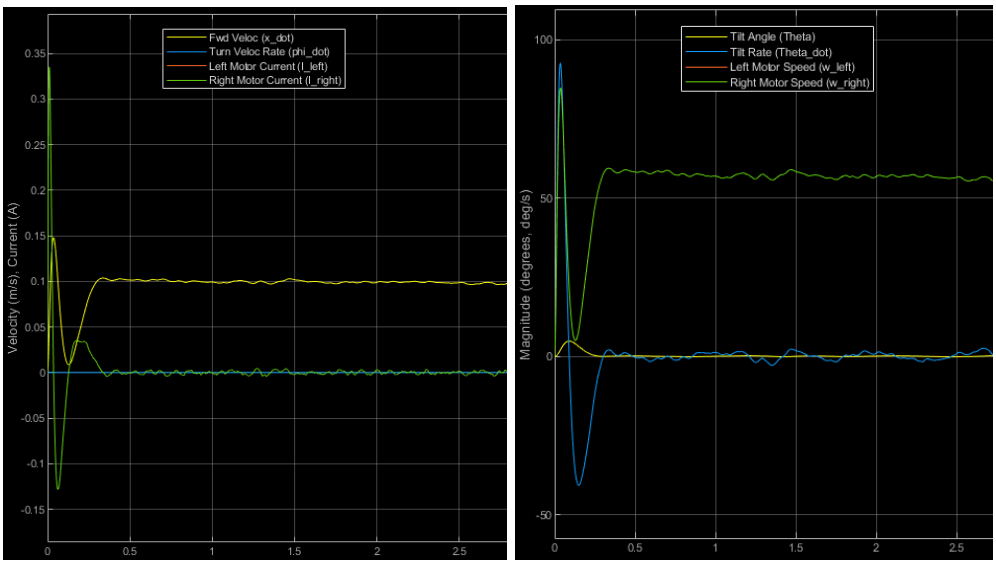


Figure9: Forward velocity controlled to 0.1m/s, tilt angle stabilized, and turn rate held constant in the presence of signal noise, with varied system parameters (Test #5)

Conclusion

To address the control problem of balancing and mobilizing a 2 wheeled robot, a 6th-order state space system was developed to model the dynamic interactions between motor inputs, individual wheel speeds, and robot pitch. Particular focus was put on accurate modelling and use of real world data, and so a CAD model was developed and motor measurements were made so that critical parameters, like inertias and robot center-of-mass could be used to build an accurate system matrix. Sensors were used to directly output 4 of the 6 state variables, but a Kalman filter observer was needed to obtain the other 2 states (motor current) and provide full state feedback. An LQR controller was designed based on desired performance parameters, however, a different manually pole-placed controller was used to provide better settling time at the cost of control effort. In the end, our system was modeled in simulink and able to consistently stabilize and move in desired directions in the presence of sensor noise and varied system parameters thanks to the robust controller and Kalman filter observer.

Appendix:

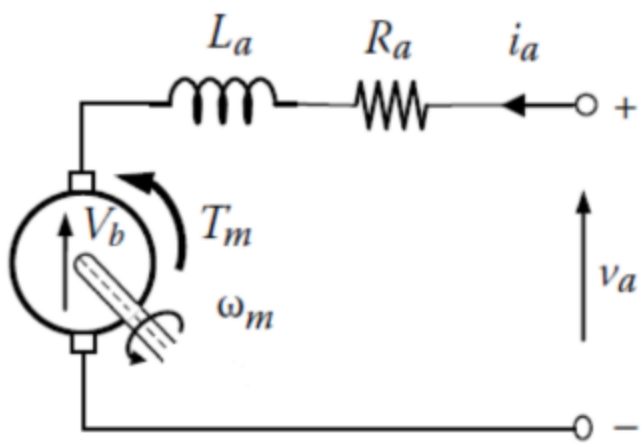
1. Parameter values

Parameter	Nominal	Units	Lower limit (%)	Upper limit (%)	Name	Source
I_{yy}	0.002153579	kgm^2	0	6	Robot Inertia about axle (Y_{rob})	From CAD mat. properties. CAD gives minimum mass and inertia, wires etc on the real robot will slightly increase this value.
I_{zz}	0.002035074	kgm^2	0	6	Robot Inertia about Z_{rob} axis	From CAD mat. properties. CAD gives minimum mass and inertia, wires etc on the real robot will slightly increase this value.
r	0.1	m	1	1	Wheel radius	From CAD model, radial tolerance is tight
m_{rob}	0.829	kg	0	6	Mass of robot	From CAD mat. properties. CAD gives minimum mass and inertia, wires etc on the real robot will slightly increase this value.
m_{wheel}	0.038	kg	2	2	Mass of wheel	From CAD mat. properties, minimal variance in manufacturing
J_{wheel}	0.000131608	kgm^2	15	15	Wheel Inertia	From CAD mat. properties, minimal variance in manufacturing. Includes rotor inertia, approximated by $\frac{1}{2}Mr^2$, where the rotor is assumed to be ½ motor mass. Could vary significantly.
$trackwidth$	0.1704	m	2	2	-	This parameter is set when assembling the robot, will be <3mm to nominal
d	0.031	m	5	5	Wheel axis to COM	From CAD mat. properties, assembly from one to another could vary, material prop. could be slightly inaccurate
g	9.81	$\frac{m}{s^2}$	0	0	Gravity	-
Kt	0.980	$\frac{Nm}{A}$	6	6	Torque Constant, back emf	From motor specification (see *Appendix 14*), could vary motor to motor, could have non-linearities
L	0.015	$Henry$	1	1	Motor Inductance	Motor was directly measured by digital multimeter, highly accurate
R	3.0	Ω	1	1	Motor	Motor was directly measured by

					Resistance	digital multimeter, highly accurate

2. Full derivation of dynamics

Drive Motor Dynamics



Applied voltage (V_a) relates to current (I_a) by a first order differential equation

$$V_a = R_a I_a + L_a \frac{dI_a}{dt} + V_b$$
$$V_b = w_m K_t$$
$$V_a = R_a I_a + L_a \frac{dI_a}{dt} + wK_t$$

where V_b is back emf voltage

Solving for $\frac{d}{dt}(I_a)$ and applying to the left and right motor circuits gives

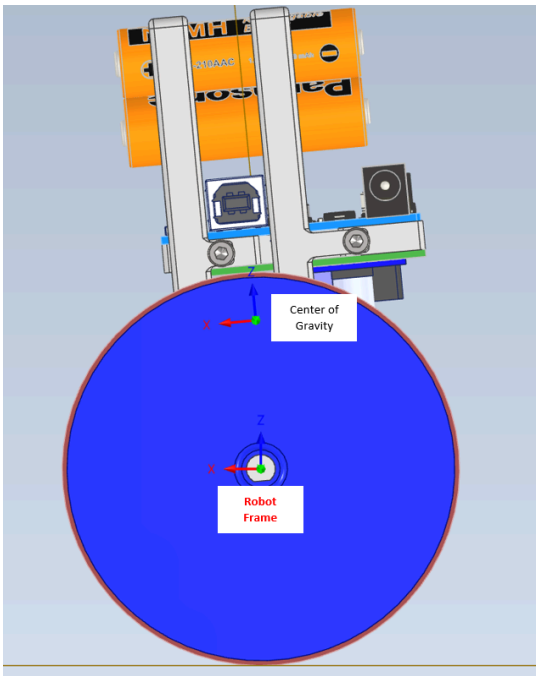
$$\frac{d}{dt}(I_{left}) = \frac{-R}{L} I_{left} - \frac{K_t}{L} w_{left} + \frac{1}{L} v_{left}$$
$$\frac{d}{dt}(I_{right}) = \frac{-R}{L} I_{right} - \frac{K_t}{L} w_{right} + \frac{1}{L} v_{right}$$

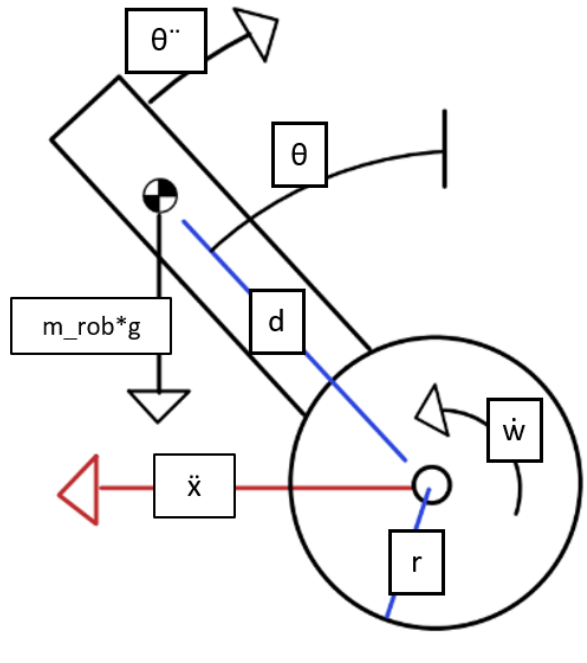
And

$$T_{right} = K_t I_{right}$$
$$T_{left} = K_t I_{left}$$

Tilt Dynamics: Body and Wheel

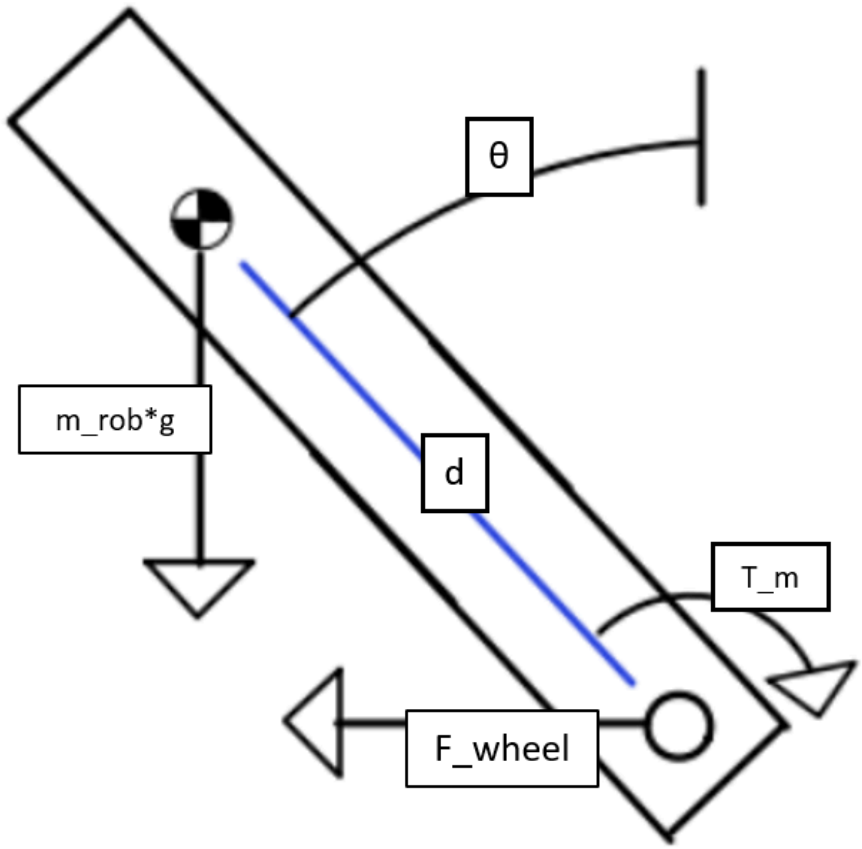
Consider a side view of the robot (XZ plane), where accelerations ($\ddot{x}$, $\ddot{\theta}$) are used to correct pitch error (θ).





A net wheel torque of T_m causes robot acceleration ($\ddot{x}$), pitching ($\ddot{\theta}$), and wheel acceleration ($\dot{w}$), and is internal to the robot body. We assume the wheel has no slip.

Body Dynamics



$$m_{rob}gdsin(\theta) - T_m + F_{wheel}dcos(\theta) = I_{yy}\ddot{\theta}$$
$$F_{wheel} = m_{rob}\ddot{x}$$
$$m_{rob}gdsin(\theta) - T_m + m_{rob}\ddot{x}dcos(\theta) = I_{yy}\ddot{\theta}$$

The robot will be operated very close to its upright position, so we make small angle approximations to linearize the second order ODE about the unstable equilibrium position.

$$sin(\theta) \simeq \theta$$
$$cos(\theta) \simeq 1$$
$$m_{rob}gd\theta - T_m + m_{rob}\ddot{x}d = I_{yy}\ddot{\theta}$$

Where T_m relates to motor currents by

$$T_m = T_{right} + T_{left} = K_t I_{right} + K_t I_{left}$$

which gives:

$$\frac{d}{dt}(\theta) = \dot{\theta}$$

$$\frac{d}{dt}(\dot{\theta}) = \frac{m_{rob}gd}{I_{yy}}\theta - \frac{K_t}{I_{yy}}I_{right} + \frac{K_t}{I_{yy}}I_{left} + \frac{m_{rob}d}{I_{yy}}\ddot{x}$$

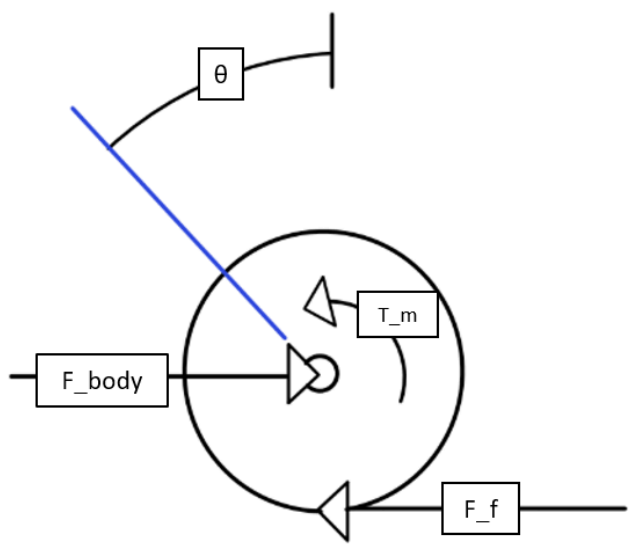
Substitute $\ddot{x}$ using the fourth state equation

$$\frac{d}{dt}(\dot{\theta}) = \frac{m_{rob}gd}{I_{yy}}\theta - \frac{K_t}{I_{yy}}I_{right} + \frac{K_t}{I_{yy}}I_{left} + \frac{m_{rob}d}{I_{yy}}\left(\frac{K_t r(I_{right} + I_{left})}{r^2 m_{rob} + r^2 m_{wheel} + J_{wheel}}\right)$$

Simplify

$$\frac{d}{dt}(\dot{\theta}) = \frac{m_{rob}gd}{I_{yy}}\theta + \left(\frac{m_{rob}dK_t r()}{I_{yy}(r^2 m_{rob} + r^2 m_{wheel} + J_{wheel})} - \frac{K_t}{I_{yy}}\right)I_{right} + \left(\frac{m_{rob}dK_t r}{I_{yy}(r^2 m_{rob} + r^2 m_{wheel} + J_{wheel})} + \frac{K_t}{I_{yy}}\right)I_{left}$$

Wheel Dynamics



Net torques cause wheel acceleration ($\dot{w}$), and net forces cause linear acceleration ($\ddot{x}$).

$$T_m - F_f r = J_{wheel}\dot{w}$$
$$F_f - F_{body} = m_{wheel}\ddot{x}$$

The robot body and wheel are connected at the wheel axis, so

$$F_{body} = F_{wheel} = m_{rob}\ddot{x}$$

Substituting the previous equations we find

$$T_m - (m_{rob}\ddot{x} + m_{wheel}\ddot{x})r = J_{wheel}\frac{\ddot{x}}{r}$$

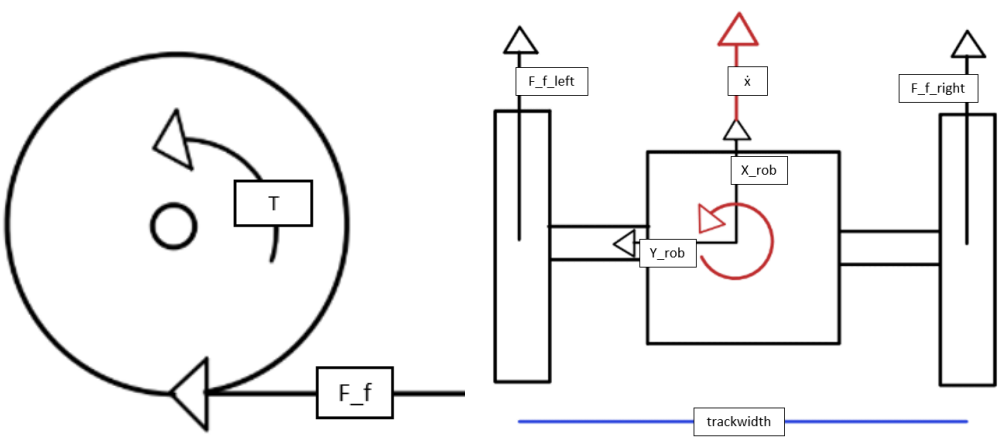
where

$$T_m = T_R + T_L = K_t I_{right} + K_t I_{left}$$

giving:

$$\frac{d}{dt}(\dot{x}) = \frac{K_t r(I_{right} + I_{left})}{r^2 m_{rob} + r^2 m_{wheel} + J_{wheel}}$$

Steering Dynamics



The yaw (ϕ) acceleration and wheel (w) acceleration are related (no slip) by

$$\frac{trackwidth}{r}\frac{d}{dt}(\phi_{dot}) = \frac{d}{dt}(w_{right} - w_{left})$$

Yaw acceleration dynamics are expressed

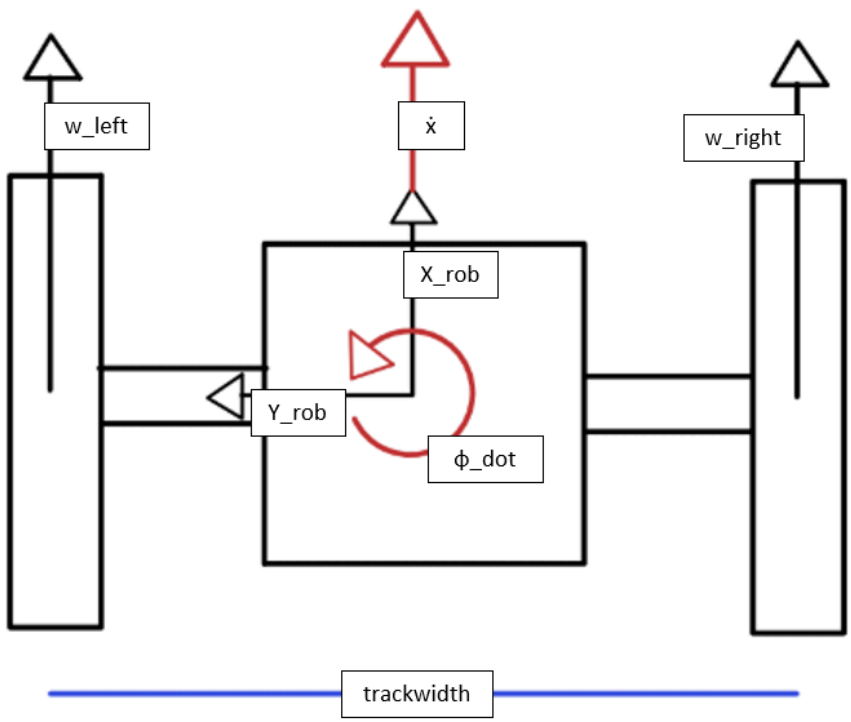
$$\frac{(T_{right} - T_{left})trackwidth}{2r} = (I_{zz} + \frac{trackwidth}{r}J_{wheel})\frac{d}{dt}(\phi_{dot})$$

Solving for $\frac{d}{dt}(\phi_{dot})$

$$\frac{d}{dt}(\phi_{dot}) = \frac{(T_{right} - T_{left})trackwidth}{2r(I_{zz} + \frac{trackwidth}{r}J_{wheel})}$$

Substitute $T = K_t I$ and simplify

$$\frac{d}{dt}(\phi_{dot}) = \frac{(K_t)trackwidth}{2r(I_{zz} + \frac{trackwidth}{r}J_{wheel})}I_{right} - \frac{(K_t)trackwidth}{2r(I_{zz} + \frac{trackwidth}{r}J_{wheel})}I_{left}$$



No slip relates wheel velocities (w_{right}, w_{left}) and robot motion ($\dot{x}$, ϕ_{dot}) by

$$\dot{x} = \frac{w_{right}r + w_{left}r}{2}$$

$$\phi_{dot} = \frac{w_{right}r - w_{left}r}{trackwidth}$$

Solving for w_{right} and w_{left} in terms of ϕ_{dot} and $trackwidth$

```
>> syms x wr r wl phi tw
>> eq1 = x==(wr*r+wl*r)/2

eq1 =

x == (r*wl)/2 + (r*wr)/2

>> eq2 = phi==(wr*r-wl*r)/tw

eq2 =

phi == -(r*wl - r*wr)/tw

>> solve(eq1,eq2,[wr,wl])

ans =

  struct with fields:

    wr: (2*x + phi*tw)/(2*r)
    wl: (2*x - phi*tw)/(2*r)
```

$$w_{left} = \frac{1}{2r}(2\dot{x} - \phi_{dot} trackwidth)$$

$$w_{right} = \frac{1}{2r}(2\dot{x} + \phi_{dot} trackwidth)$$

Take time derivative of w_{left} and w_{right}

$$\dot{w}_{left} = \frac{1}{2r}(2\ddot{x} - \frac{d}{dt}(\phi_{dot}) trackwidth)$$

$$\dot{w}_{right} = \frac{1}{2r}(2\ddot{x} + \frac{d}{dt}(\phi_{dot}) trackwidth)$$

$\frac{d}{dt}(\dot{x})$ and $\frac{d}{dt}(\phi_{dot})$ dynamics are previously derived. Substitute:

$$\dot{w}_{left} = \frac{1}{2r}\left(\frac{2K_t r(I_{right} + I_{left})}{r^2 m_{rob} + r^2 m_{wheel} + J_{wheel}} - \frac{(K_t) trackwidth}{2r(I_{zz} + \frac{trackwidth}{r} J_{wheel})} I_{right} + \frac{(K_t) trackwidth}{2r(I_{zz} + \frac{trackwidth}{r} J_{wheel})} I_{left}\right)$$

$$\dot{w}_{right} = \frac{1}{2r}\left(\frac{2K_t r(I_{right} + I_{left})}{r^2 m_{rob} + r^2 m_{wheel} + J_{wheel}} + \frac{(K_t) trackwidth}{2r(I_{zz} + \frac{trackwidth}{r} J_{wheel})} I_{right} - \frac{(K_t) trackwidth}{2r(I_{zz} + \frac{trackwidth}{r} J_{wheel})} I_{left}\right)$$

3. Open loop system eigenvalues

```
%eig of system with no feedback
EigenValues_OpenLoop = eig(A)
```

```
EigenValues_OpenLoop = 6×1 complex
10^2 x
     0.0000 + 0.1082i
     0.0000 - 0.1082i
    -1.0000 + 5.9844i
    -1.0000 - 5.9844i
    -1.0000 + 1.1200i
    -1.0000 - 1.1200i
```

4. Controllability and Observability Matrices, Rank

```
%Define Controllability Matrix and Observability Matrix
CAB=[];
for i =0:5
    for ii = 1:2
    CAB = [CAB, A^i*B(:,ii)];
    end
end
OAC =[];
for i = 0:5  % from C*A^0 to C*A^9
    OAC = [OAC; C*A^i];
end
CAB
```

```
CAB = 6×12
10^16 x
         0         0         0         0    0.0000   -0.0000   -0.0000    0.0000   -0.0000    0.0000 ...
         0         0    0.0000   -0.0000   -0.0000    0.0000   -0.0000    0.0000    0.0004   -0.0004
    0.0000         0   -0.0000         0   -0.0000    0.0000    0.0000   -0.0000    0.0003   -0.0003
         0    0.0000         0   -0.0000    0.0000   -0.0000   -0.0000    0.0000   -0.0003    0.0003
         0         0    0.0000   -0.0000   -0.0000    0.0000   -0.0000    0.0000    0.0026   -0.0026
         0         0   -0.0000    0.0000    0.0000   -0.0000    0.0000   -0.0000   -0.0026    0.0026
```

```
OAC
```

```
OAC = 24×6
10^14 x
    0.0000         0         0         0         0         0
         0    0.0000         0         0         0         0
         0         0         0         0    0.0000         0
         0         0         0         0         0    0.0000
         0    0.0000         0         0         0         0
   -0.0000         0    0.0000   -0.0000         0         0
         0         0    0.0000   -0.0000         0         0
         0         0   -0.0000    0.0000         0         0
   -0.0000         0    0.0000   -0.0000         0         0
         0   -0.0000   -0.0000    0.0000   -0.0000    0.0000
       ⋮
```

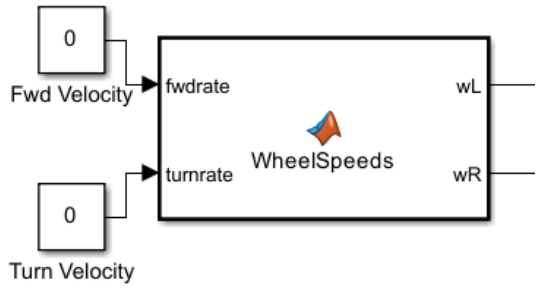
```
ObservableStates = rank(OAC)
```

```
ObservableStates = 6
```

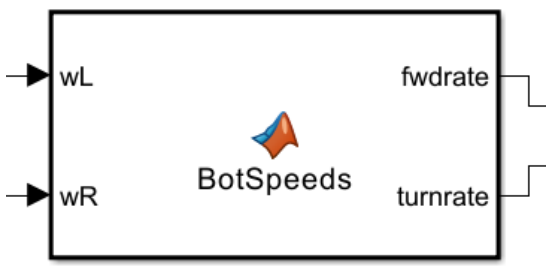
```
ControllableStates = rank(CAB)
```

```
ControllableStates = 6
```

5. WheelSpeeds & BotSpeeds Function




```
function [wL, wR] = WheelSpeeds(fwdrate, turnrate)
r=0.1;
trackwidth = 0.1704;
wL = 1/(2*r)*(2*fwdrate - turnrate*trackwidth);
wR = 1/(2*r)*(2*fwdrate + turnrate*trackwidth);
end
```




```
function [fwdrate, turnrate] = BotSpeeds(wL, wR)
r=0.1;
trackwidth = 0.1704;

fwdrate = (wR*r + wL*r)/2
turnrate = (wR*r - wL*r)/trackwidth

end
```

6. Pole placing, then matching gains for left and right feedback

```
%Test Different Pole Locations for Closed loop system
K = place(A,B,[-100 -110 -4 -4 -500 -500])
```

```
K = 2×6
    261.7613   -3.0725   10.3625   -2.7185    3.6847    3.3226
    254.9338   -2.9829    5.6616    1.9075    4.5383    2.2619
```

→

```
K = [260 -3.0 10.5 -3.0 4.0 3.0; 260 -3.0 10.5 -3.0 4.0 3.0]
```

```
K = 2×6
    260.0000   -3.0000   10.5000   -3.0000    4.0000    3.0000
    260.0000   -3.0000   10.5000   -3.0000    4.0000    3.0000
```

```
%Actual Eigenvalues
eig(A-B*K)
```

```
ans = 6×1 complex
10^2 x
   -4.7919 + 0.0000i
   -1.3348 + 0.0000i
   -0.8329 + 0.0000i
   -0.0403 + 0.0000i
   -1.0000 + 5.9844i
   -1.0000 - 5.9844i
```

7. Kref Calculation for setting desired steady-state outputs

```
%Find Kref, so we can set desired values in simulink
Z = -C*inv(A-B*K)*B;
Kref = inv(Z.'*Z)*Z.'
```

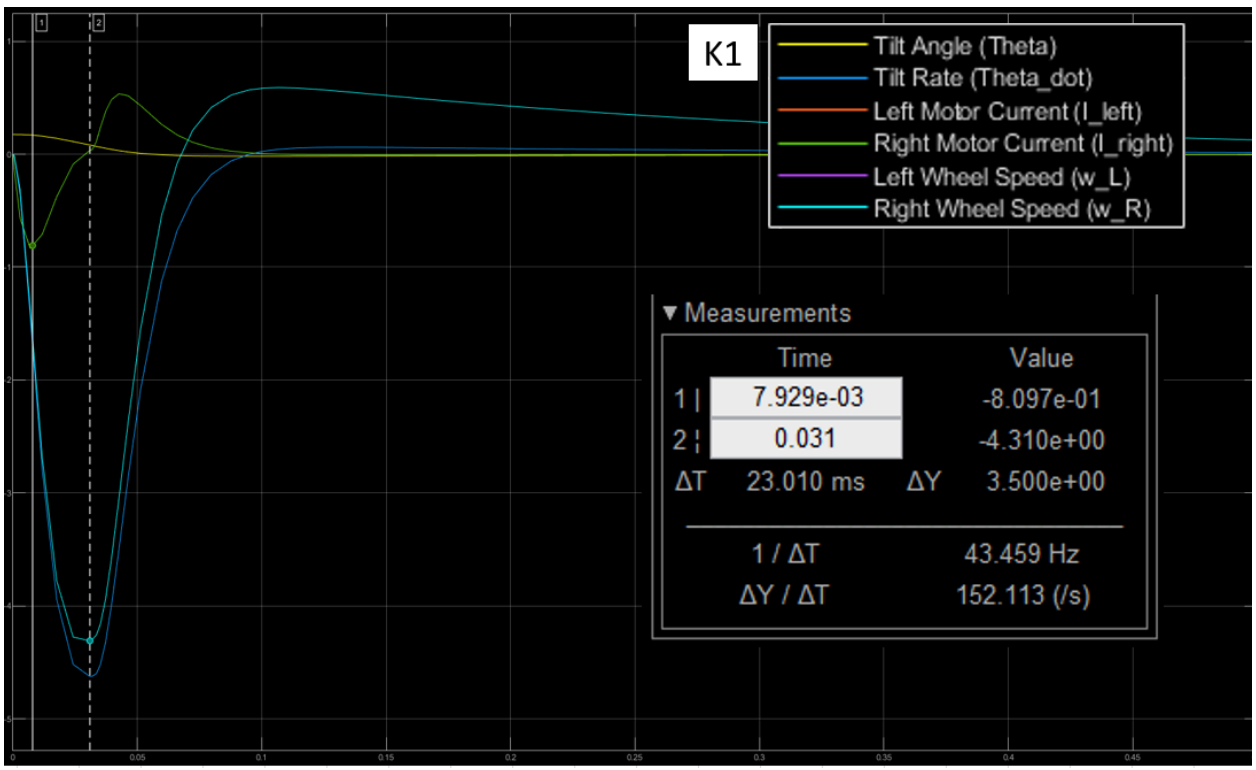
```
Kref = 2×4
    0.0000         0    4.9800    3.0000
    0.0000         0    4.0000    3.9800
```

8. Pole Placed Controller Performance and Plots

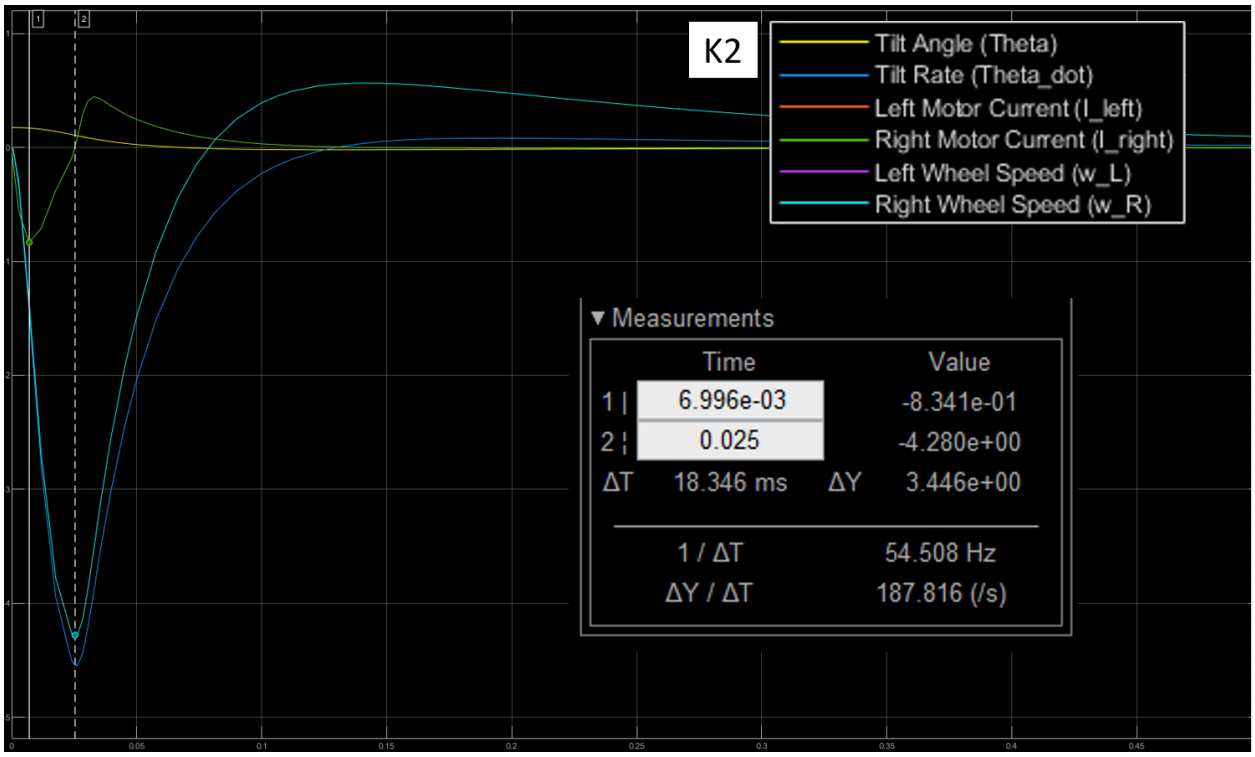
Feedback Gain K	Initial Condition	$T_{S,\theta}$ (s)	$T_{S,\theta_{dot}}$ (s)	w Max (rad/s)	I Max (A)	EigenValues (Poles)
K1 [260 -3 10.5 -3 4 3] [260 -3 10.5 -3 4 3]	$\theta = 10°$	0.055	0.2	-4.31	0.82	10^2x [-4.79] [-1.33] [-0.83] [-0.04] [-1.0+6j] [-1.0-6j]
K2 [315 -3.6 11.5 -3.6 6.2 4.8] [315 -3.6 11.5 -3.6 6.2 4.8]	$\theta = 10°$	0.081	0.25	-4.28	0.834	10^2x [-3.33] [-3.33] [-0.35] [-0.05] [-1.0+6j] [-1.0-6j]
K3 [961 -20 13 -5 16 12.5] [961 -20 13 -5 16 12.5]	$\theta = 10°$	0.046	0.10	-4.31	1.019	10^2x [-3.01] [-3.01] [-1.27] [-0.04] [-1.0+6j] [-1.0-6j]
K4 [11 -0.5 0.5 -1 -0.5 0.4] [11 -0.5 0.5 -1 -0.5 0.4]	$\theta = 10°$	0.144	0.25	-3.9	0.53	10^2x [-0.93] [-0.29] [-0.29] [-0.16] [-1.0+6j] [-1.0-6j]

K5 [11 -0.5 0.75 -1.2 -0.18 0.5] [11 -0.5 0.75 -1.2 -0.18 0.5]	$\theta = 10°$	0.20	0.35	-2.37	0.52	10^2x [-0.72+.36j] [-0.72-.36j] [-0.13+.14j] [-0.13-.14j] [-1.0 + 4.8j] [-1.0 - 4.8j]

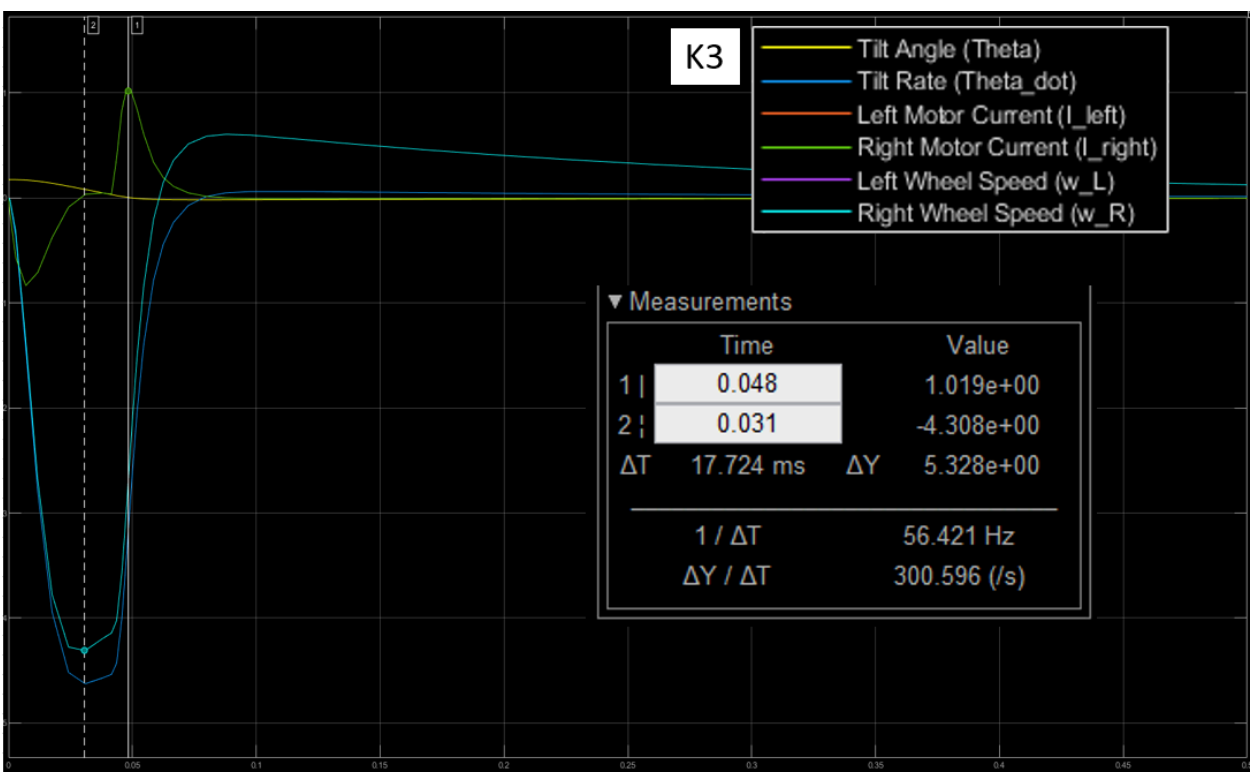
K1 Controller:



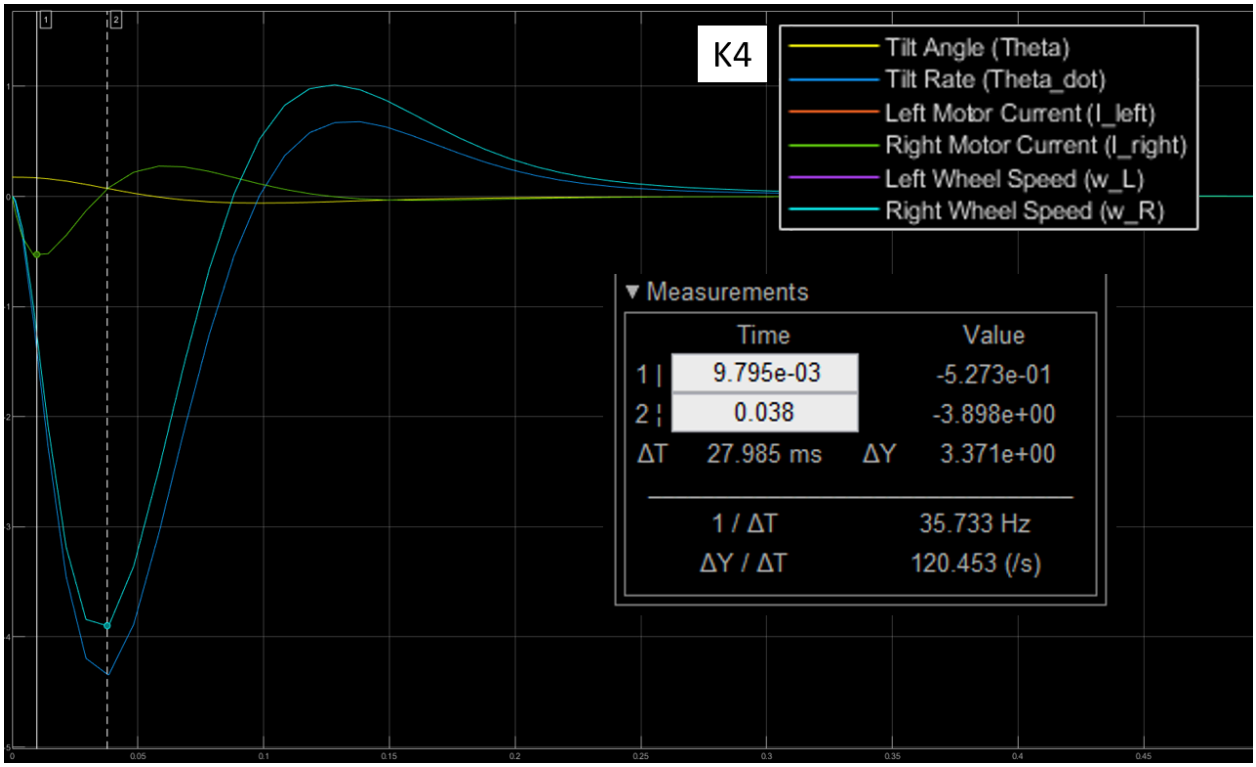
K2 Controller:



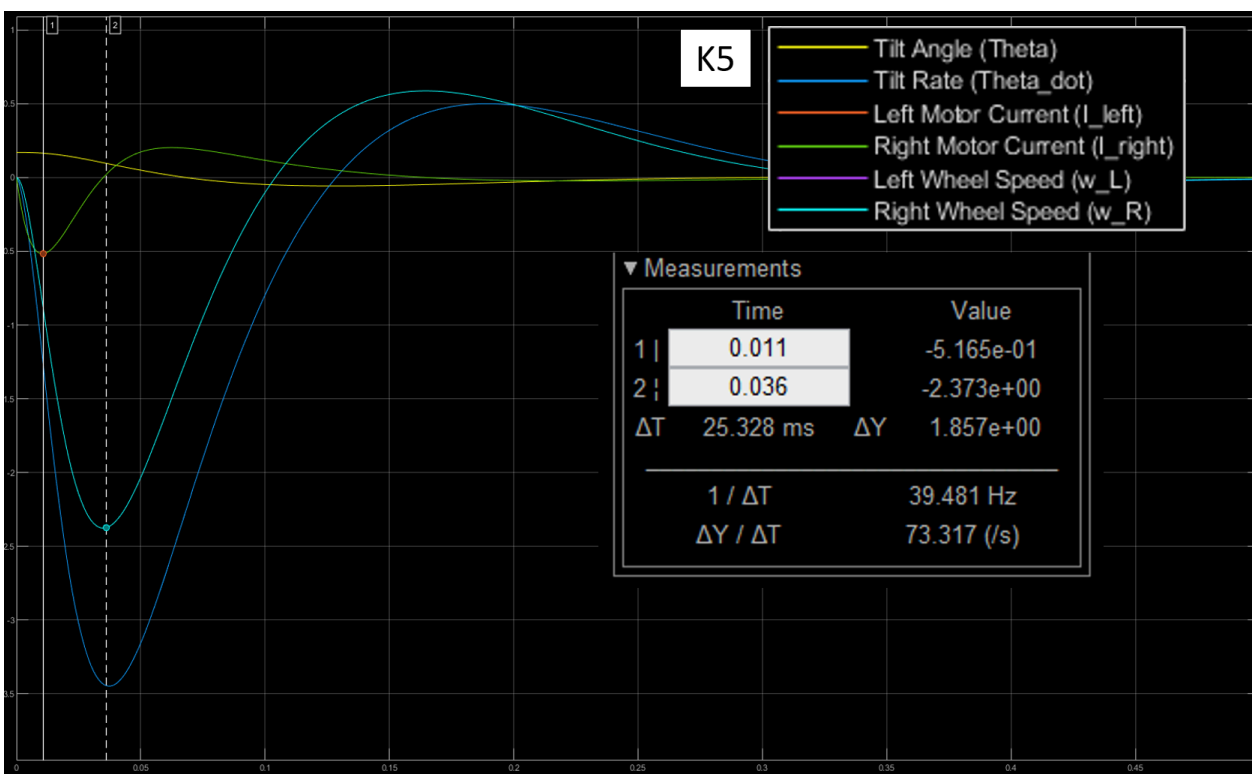
K3 Controller:



K4 Controller:



K5 Controller:



9. LQR Controller Design Script & Plots

```
for rho = [10,1,0.1,0.01,0.001]
```

```
%Find LQR with rho value
```

```
   %Define max values allowed for state vars 1, 2, ...
   xMax = [1 5 1 1 4 4]
   ts = [0.1 0.1 0.25 0.25 0.25 0.25]
   us = [4 4]
```

```
   % State cost weighting
       Q_LQR = rho*diag((1 ./ xMax).^2 .* (1 ./ ts));
   % Input costweighting
       R_LQR = diag((1 ./ us).^2);
   %Use icare to solve the alg ricatti eqn
       [M_LQR K_LQR l_LQR] = icare(A,B,Q_LQR,R_LQR);
```

```
   %M is soln to ARE (performance matrix), Optimal K!:
       K_LQR = inv(R_LQR)*B.'*M_LQR
   %Eigenvalues of closed system
       eig(A-B*K_LQR)
```

```
%Plot LQR system
   %Define initial state, and system
```

```
        initial_state = [.1745; 0; 0; 0; 0; 0];      %10 degree tilt IC

        sys = ss(A-B*K_LQR, B, C, 0);

    %Solve with initial condition

        [y t x] = initial(sys, initial_state);



    plot(t,x(:,[1 2 3 4 5 6],1))

    xlim([0 1])

    ylim([-2 1])


%Title

    if rho==100

        title('Closed Loop Step Response: LQR, rho=100');

    elseif rho==10

        title('Closed Loop Step Response: LQR, rho=10');

    elseif rho==1

        title('Closed Loop Step Response: LQR, rho=1');

    elseif rho==0.1

        title('Closed Loop Step Response: LQR, rho=0.1');

    elseif rho==0.01

        title('Closed Loop Step Response: LQR, rho=0.01');

    end

%Labels

    xlabel('Time (seconds)');

    ylabel('Amplitude');

    legend('theta','theta_dot','I_left','I_right','w_left','w_right')

    grid on;

    figure;

end
```

```
xMax = 1×6

    1     5     1     1     4     4

ts = 1×6

   0.1000    0.1000    0.2500    0.2500    0.2500    0.2500

us = 1×2

    4     4

K_LQR = 2×6

  73.8644    3.8100   28.2836   -4.4834    5.3866    0.6533

  36.5091    2.2713   -4.4834   27.9446   -0.6533    5.3866

ans = 6×1 complex

10^3 ×

 -1.6878 + 0.0000i

 -1.1863 + 0.4544i

 -1.1863 - 0.4544i

 -0.0800 + 0.0000i

 -0.0040 + 0.0080i

 -0.0040 - 0.0080i
```

```
xMax = 1×6

    1     5     1     1     4     4

ts = 1×6

   0.1000    0.1000    0.2500    0.2500    0.2500    0.2500

us = 1×2

    4     4

K_LQR = 2×6

  23.4750    1.1580    9.1839   -2.6448    1.2377    0.2059
```

```
  11.9202    0.7103   -2.6448    8.9418   -0.2059    1.2377

ans = 6×1 complex

10^2 ×

 -4.8998 + 5.6506i

 -4.8998 - 5.6506i

 -5.3681 + 0.0000i

 -0.8384 + 0.0000i

 -0.0389 + 0.0832i

 -0.0389 - 0.0832i





xMax = 1×6

    1     5     1     1     4     4

ts = 1×6

   0.1000    0.1000    0.2500    0.2500    0.2500    0.2500

us = 1×2

    4     4

K_LQR = 2×6

   7.6391    0.2759    2.2394   -0.8924    0.1850    0.0555

   4.3921    0.1931   -0.8924    2.1484   -0.0555    0.1850

ans = 6×1 complex

10^2 ×

 -2.0271 + 4.9831i

 -2.0271 - 4.9831i

 -1.4062 + 0.1163i

 -1.4062 - 0.1163i

 -0.0293 + 0.0962i
```

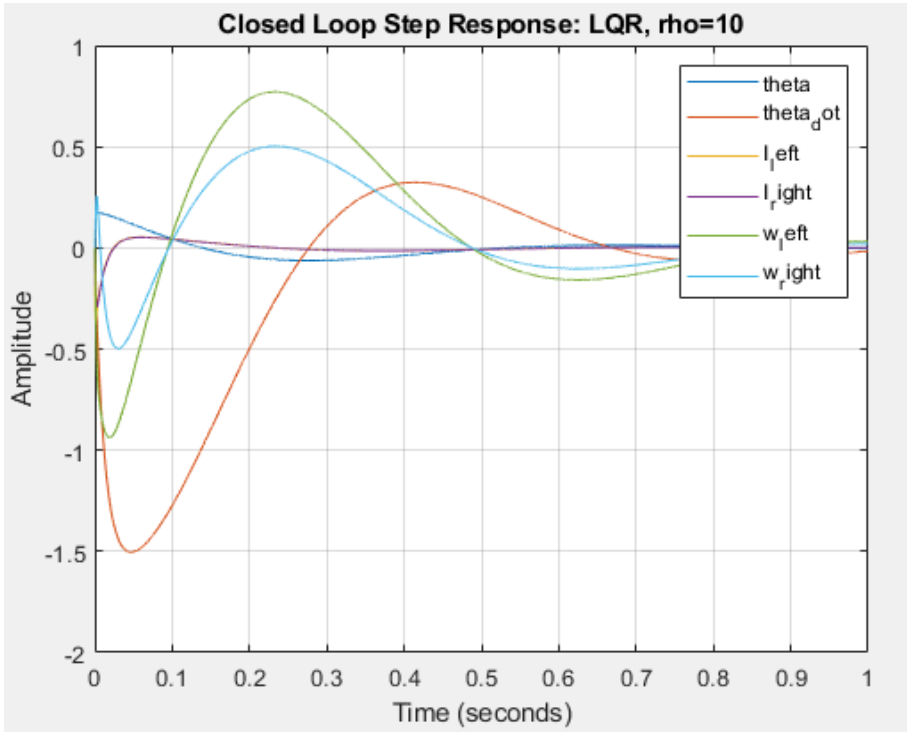
```
 -0.0293 - 0.0962i
```

```
xMax = 1×6

    1     5     1     1     4     4

ts = 1×6

   0.1000    0.1000    0.2500    0.2500    0.2500    0.2500

us = 1×2

    4     4

K_LQR = 2×6

   2.4732    0.0516    0.3352   -0.1391    0.0201    0.0113

   1.5663    0.0424   -0.1391    0.3144   -0.0113    0.0201

ans = 6×1 complex

10^2 ×

 -1.1541 + 4.8306i

 -1.1541 - 4.8306i

 -1.0499 + 0.6446i

 -1.0499 - 0.6446i

 -0.0126 + 0.1062i

 -0.0126 - 0.1062i
```

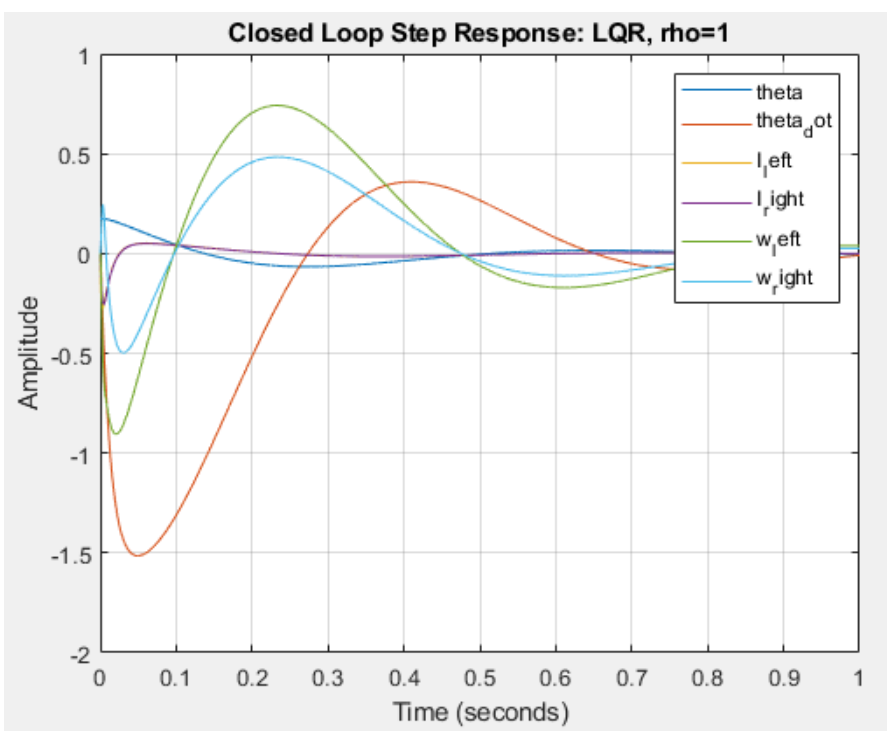
```
xMax = 1×6

    1     5     1     1     4     4

ts = 1×6
```

```
   0.1000    0.1000    0.2500    0.2500    0.2500    0.2500

us = 1×2

    4     4

K_LQR = 2×6

   0.7875    0.0115    0.0415   -0.0115    0.0020    0.0026

   0.5108    0.0104   -0.0115    0.0361   -0.0026    0.0020

ans = 6×1 complex

10^2 ×

 -1.0166 + 4.8129i

 -1.0166 - 4.8129i

 -1.0051 + 0.6716i

 -1.0051 - 0.6716i

 -0.0042 + 0.1080i

 -0.0042 - 0.1080i
```

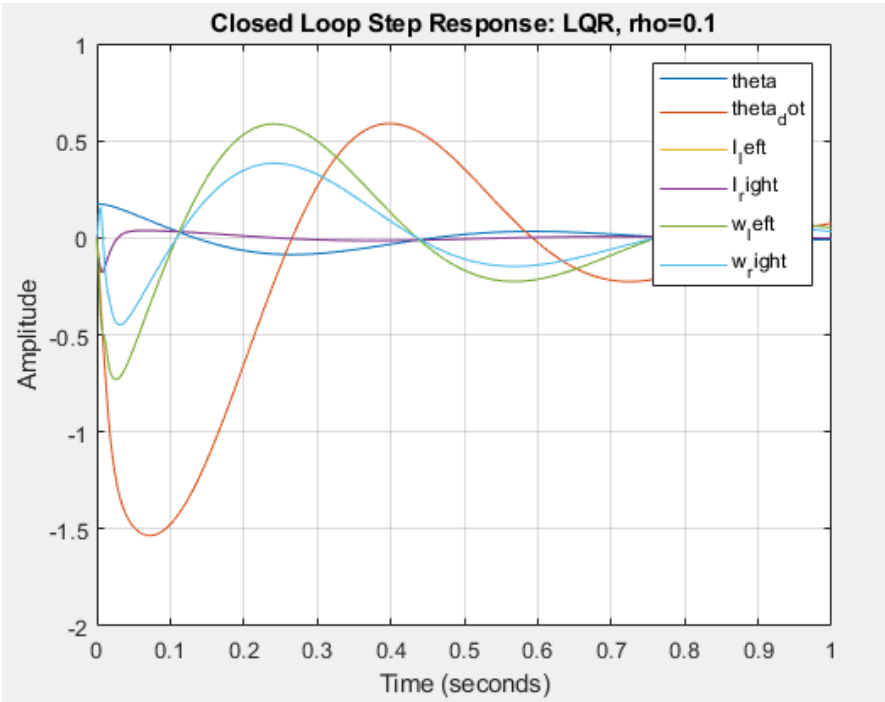
K_{LQR} for $\varphi = 10$



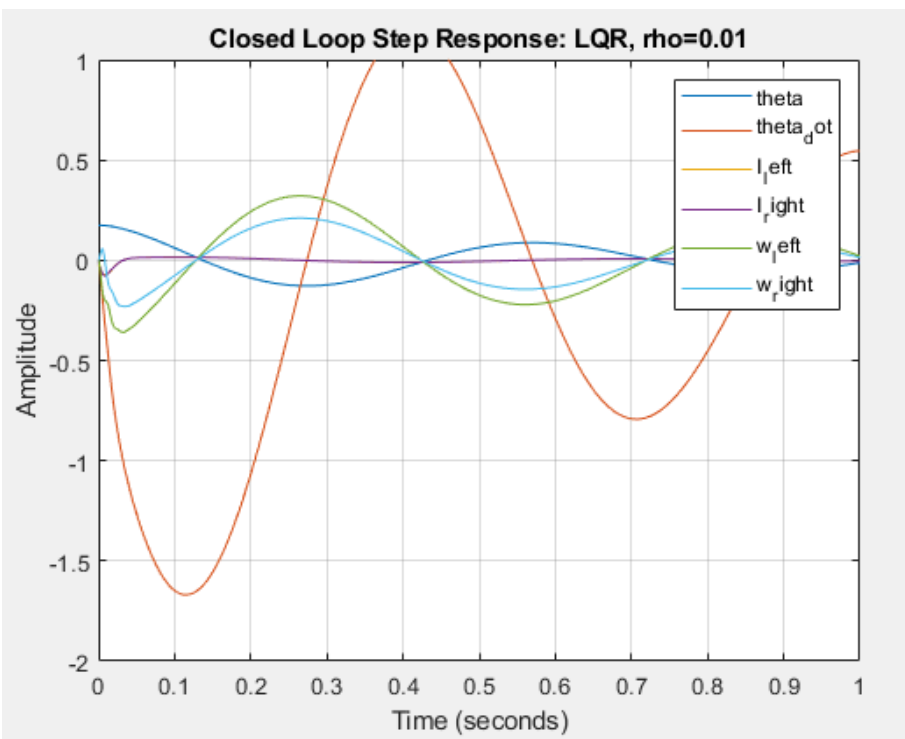
K_{LQR} for $\varphi = 1$



K_{LQR} for $\varphi = 0.1$



K_{LQR} for $\varphi = 0.01$



10. Pole-Placement Observer Design

```
%Design Observer using the dual system. Same
%as controller design, but A->AT, B->CT, C->BT (transposes)

%write desired characteristic equation
obs_poles = poles*25;
K_obsT=place(A.',C.', obs_poles)
```

```
K_obsT = 4×6
10^4 ×
   0.3661   -0.0227    0.0000   -0.0000    0.0079    0.0052
  -0.0109    0.1066    0.5450   -0.0309   -0.2061    0.4947
   0.0079    0.4648    0.8897   -0.7679    0.2654    0.6098
   0.0052   -0.5252   -0.2113    1.0955   -0.8071    0.1386
```

```
K_obs = K_obsT.'
```

```
K_obs = 6×4
10^4 ×
   0.3661   -0.0109    0.0079    0.0052
  -0.0227    0.1066    0.4648   -0.5252
   0.0000    0.5450    0.8897   -0.2113
  -0.0000   -0.0309   -0.7679    1.0955
   0.0079   -0.2061    0.2654   -0.8071
   0.0052    0.4947    0.6098    0.1386
```

```
%A is same as system!
B_obs = [B K_obs];
C_obs = eye(6);  % outputs all estimated states
D_obs = zeros(6, size(B_obs,2));
```

11. Kalman Filter Observer Design

```
%Design Observer using the dual system. Same
%as controller design, but A->AT, B->CT, C->BT (transposes)
```

```
%Kalman Obs
Qn = 0.0001*eye(6);     % process noise
Rn = 0.001*eye(4);      % measurement noise
%Use lqe commmand to determine Kalman Gain
L_obs = lqe(A_nominal, eye(6), C, Qn, Rn);
```

```
%write desired characteristic equation
obs_poles = poles*20;
K_obsT=place(A_nominal.',C.', obs_poles)
K_obs = K_obsT.'
%A is same as system!
B_obs = [B_nominal L_obs];
C_obs = eye(6);  % outputs all estimated states
D_obs = zeros(6, size(B_obs,2));
```

12. Robustness Testing Script

```
%Parameters. 1st element is nominal, 2nd element is low limit(%), 3rd is
%upper limit(%), values are in appendix15
```

```
Iyy = [0.002153579, 0, .06];      %2153579gmm^2 converted to kgm^2, from
CAD model no wheels, this checks out
```

```
Izz = [0.002035074, 0, .06];      %2035074gmm^2, from CAD model w/wheels

r = [0.1, .01, .01];               %from CAD model

m_rob = [0.829, 0, .06];          %829.744 g from CAD model

m_wheel = [0.038, .02, .02];        %0.038kg from CAD

J_wheel = [0.000031608+0.0001, .15, .15];  %31.608 kg mm^2 from CAD abt
rotation axis + inertia of motor (.5M*R^2, M=100g R=30mm)

trackwidth = [0.1704, .02, .02];    %170.400000000 mm from CAD

d = [0.031, .05, .05];              %31.008110000 mm from CAD

g = [9.81, 0.06, 0.06];               %m/s^2, gravity

Kt = [.980, .1, .1];             %980Nmm/A, from Nidec spec sheet

L = [0.015, .01, .01];             %1.5mH guess, Folarin will measure
actual motor

R = [3.0, .01, .01];                  %Folarin measured actual motor
```

```
%Repeated Expressions

elem1 = (Kt(1)*r(1))/(r(1)^2*m_rob(1)+r(1)^2*m_wheel(1)+J_wheel(1))

elem2 =
(Kt(1)*trackwidth(1))/(2*r(1)*(Izz(1)+(trackwidth(1)*J_wheel(1)/r(1))))
```

```
%When testing the system, the observer A matrix should not be modified.

%That is part of our design.

%Only the system itself is modified to check if the controller/observer
is

%robust A_observer uses nominal values. A is calculated using TestMatrix

%modifications (parameter ranges)
```

```
A_nominal = [0, 1, 0, 0, 0, 0; ...

   -m_rob(1)*g(1)*d(1)/Iyy(1), 0,
(m_rob(1)*d(1)*Kt(1)*r(1))/(Iyy(1)*(r(1)^2*m_rob(1)+r(1)^2*m_wheel(1)+J_
wheel(1)))+(Kt(1)/Iyy(1)),
```

```
(m_rob(1)*d(1)*Kt(1)*r(1))/(Iyy(1)*(r(1)^2*m_rob(1)+r(1)^2*m_wheel(1)+J_
wheel(1)))-(Kt(1)/Iyy(1)), 0, 0; ...
   0, 0, -R(1)/L(1), 0, -Kt(1)/L(1), 0; ...
   0, 0, 0, -R(1)/L(1), 0, -Kt(1)/L(1); ...
   0, 0, 1/(2*r(1))*(2*elem1+elem2), 1/(2*r(1))*(2*elem1-elem2), 0, 0;
...
   0, 0, 1/(2*r(1))*(2*elem1-elem2), 1/(2*r(1))*(2*elem1+elem2), 0, 0;
...
   ];
B_nominal = [0 0;
   0 0;
   1/L(1) 0;
   0 1/L(1);
   0 0;
   0 0;
   ];
%Robustness Testing Script. Uses random number generation to test
%combinations of lower(-1) nominal(0) and upper(1) parameter values,
%with the last three columns specifying IC, target x_dot, target phi_dot
%Manually type test number, run MATLAB, run Simulink, then increment
%TestNumber and repeat to see if it's robust!
```

```
TestNumber = 6;
TestMatrix = [0 0 0 0 0 0 0 0 0 0 0 0 1 0 0; ...  %nominal, IC
0 0 0 0 0 0 0 0 0 0 0 0 0 1 0; ...  %nominal, targ x_dot
0 0 0 0 0 0 0 0 0 0 0 0 0 0 1; ...  %nominal, targ phi_dot
1 1 1 1 1 1 1 1 1 1 1 1 1 0 0; ...                %upper, all
-1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 0 1 0; ... %lower
-1  0 -1  0  1 -1  1 -1 -1  0  1  1  1  0 1; ...  %random->
```

```
0 -1 -1 -1  0  1 -1 -1 -1  1  1  1  1  0 0; ...
1 -1 -1  0  0  0  1  1 -1  0  0  1  1  0 0; ...
1  1  1  0 -1  0  1 -1 -1 -1  1  0  0  1 -1; ...
-1 -1 -1 -1  0 -1  0  0  0 -1  1  0  1  1 1];
```

```
ParamMatrix = [Iyy; Izz; r; m_rob; m_wheel; J_wheel; trackwidth; d; g;
Kt; L; R];
vector = [];
for i =1:12
   if TestMatrix(TestNumber,i) == -1
     vector(i) = ParamMatrix(i,1) - ParamMatrix(i,2)*ParamMatrix(i,1);
   elseif TestMatrix(TestNumber,i) == 0
     vector(i) = ParamMatrix(i,1);
   elseif TestMatrix(TestNumber,i) == 1
     vector(i) = ParamMatrix(i,1) + ParamMatrix(i,3)*ParamMatrix(i,1);
   end
```

```
end
```

```
%Reassign Values
Iyy=vector(1)
Izz=vector(2)
r=vector(3)
m_rob= vector(4)
m_wheel= vector(5)
J_wheel=vector(6)
trackwidth= vector(7)
d= vector(8)
```

```
g= vector(9)

Kt= vector(10)

L= vector(11)

R = vector(12)
```

```
%Set ICs and Target Values

for i=13:15

   if TestMatrix(TestNumber,13) == 0

       init_angle = 0;

   elseif TestMatrix(TestNumber,13) == 1

       init_angle = 0.17;  %10degrees

   end

   if TestMatrix(TestNumber,14) == 0

       targ_fwd_veloc = 0;

   elseif TestMatrix(TestNumber,14) == 1

       targ_fwd_veloc = 0.1;  %m/s

   end

   if TestMatrix(TestNumber,15) == 0

       targ_turn_veloc = 0;

   elseif TestMatrix(TestNumber,15) == 1

       targ_turn_veloc = 0.1;  %rad/s

   end

end
```

```
%Repeated Expressions

elem1 = (Kt*r)/(r^2*m_rob+r^2*m_wheel+J_wheel)

elem2 = (Kt*trackwidth)/(2*r*(Izz+(trackwidth*J_wheel/r)))
```

```
%System Matrix

A = [0, 1, 0, 0, 0, 0; ...

   -m_rob*g*d/Iyy, 0,
(m_rob*d*Kt*r)/(Iyy*(r^2*m_rob+r^2*m_wheel+J_wheel))+(Kt/Iyy),
(m_rob*d*Kt*r)/(Iyy*(r^2*m_rob+r^2*m_wheel+J_wheel))-(Kt/Iyy), 0, 0; ...

   0, 0, -R/L, 0, -Kt/L, 0; ...

   0, 0, 0, -R/L, 0, -Kt/L; ...

   0, 0, 1/(2*r)*(2*elem1+elem2), 1/(2*r)*(2*elem1-elem2), 0, 0; ...

   0, 0, 1/(2*r)*(2*elem1-elem2), 1/(2*r)*(2*elem1+elem2), 0, 0; ...

   ];
```

```
%row 7 and 8 were 0, 0, 0, 0, 0, 0, -R/L, 0, -Kt/L, 0; ... 0, 0, 0, 0,
0,

%0, 0, -R/L, 0, -Kt/L; ..., nuked back emf term to make it linear

B = [0 0;

   0 0;

   1/L 0;

   0 1/L;

   0 0;
```

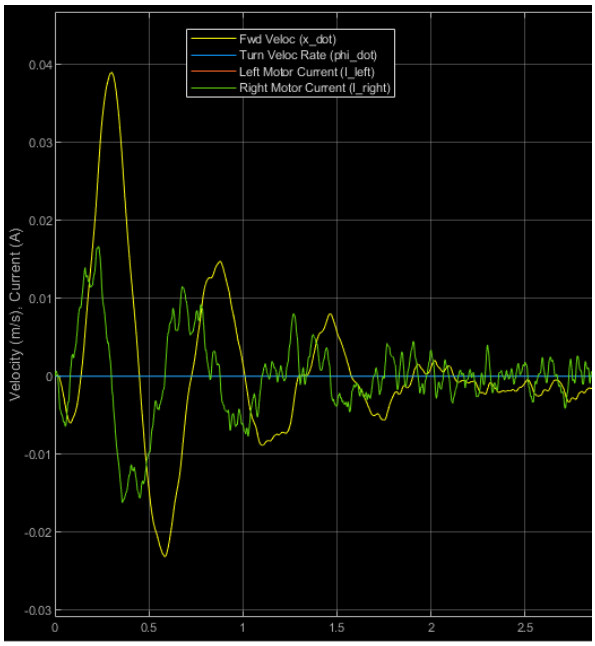
```
   0 0;

   ];
```

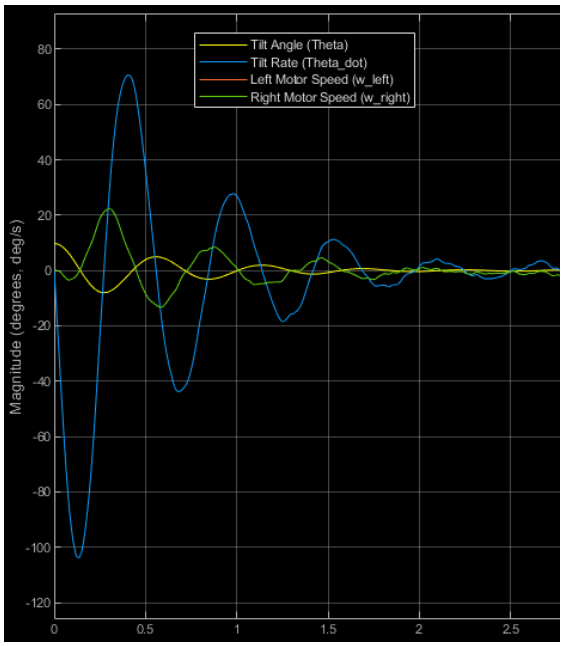
```
C = [1 0 0 0 0 0;

   0 1 0 0 0 0;

   0 0 0 0 1 0;

   0 0 0 0 0 1];

D = [0 0; 0 0; 0 0; 0 0];
```

13. Robustness Testing Matrix & Results

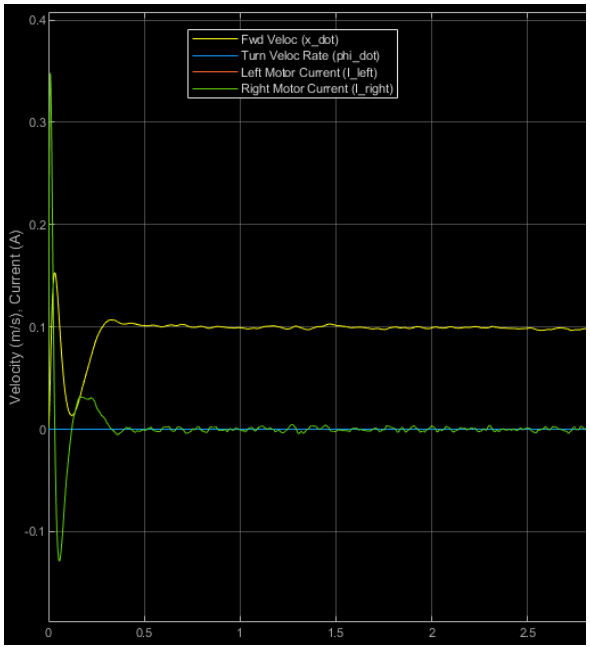
Test #	I_{yy}	I_{zz}	r	m_{rob}	m_{wheel}	J_{wheel}	$track\ width$	d	g	Kt	L	R	$\theta_{initial}$	$\dot{x}_{target}$	$\phi_{dot\ target}$
1	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
2	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0
3	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
4	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
5	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	0	1	0
6	-1	0	-1	0	1	-1	1	-1	-1	0	1	1	1	0	1

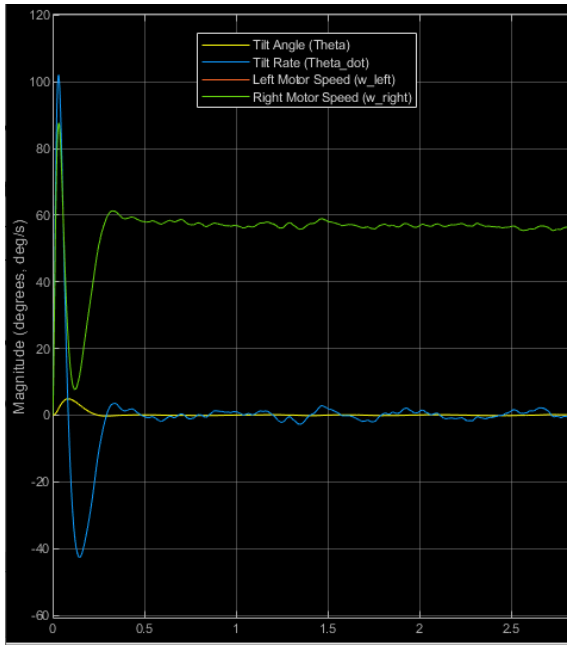
Test#1:


Fwd Veloc (x_dot)
Turn Veloc Rate (phi_dot)
Left Motor Current (I_left)
Right Motor Current (I_right)
Velocity (m/s), Current (A)

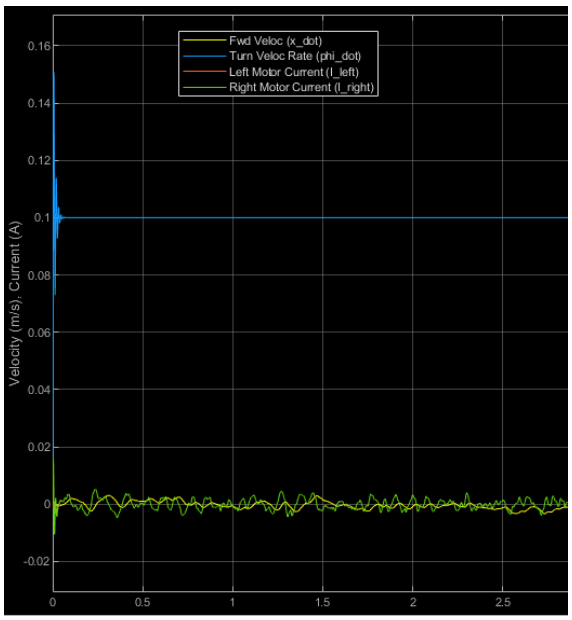

Tilt Angle (Theta)
Tilt Rate (Theta_dot)
Left Motor Speed (w_left)
Right Motor Speed (w_right)
Magnitude (degrees, deg/s)

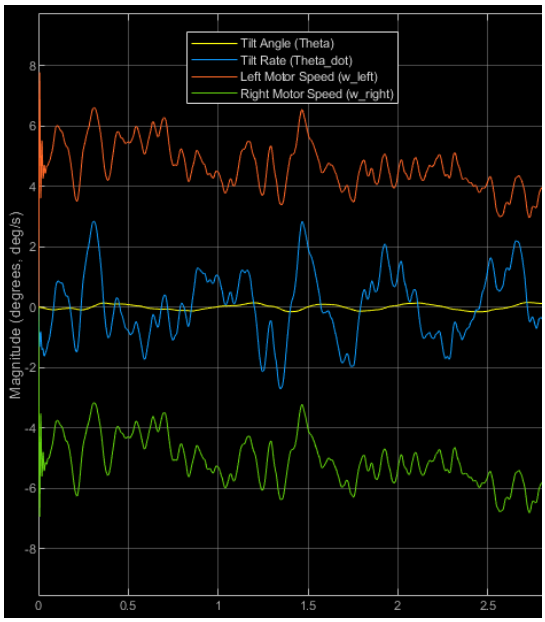
Test#2:


Fwd Veloc (x_dot)
Turn Veloc Rate (phi_dot)
Left Motor Current (I_left)
Right Motor Current (I_right)
Velocity (m/s), Current (A)

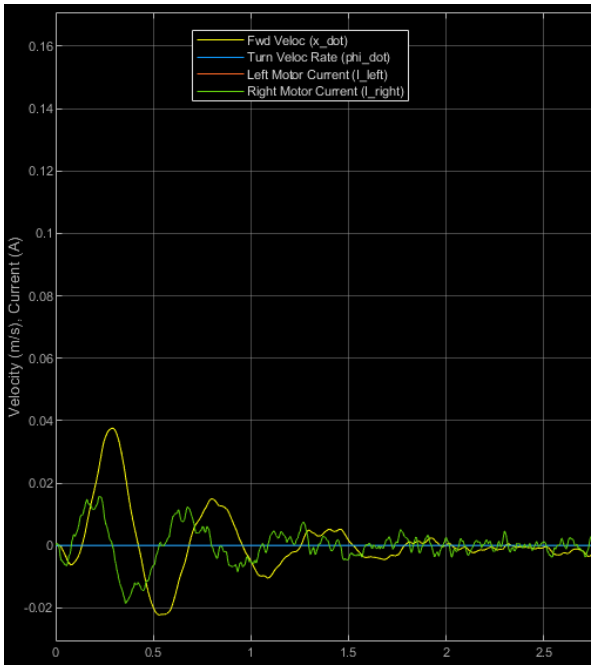

Tilt Angle (Theta)
Tilt Rate (Theta_dot)
Left Motor Speed (w_left)
Right Motor Speed (w_right)
Magnitude (degrees, deg/s)

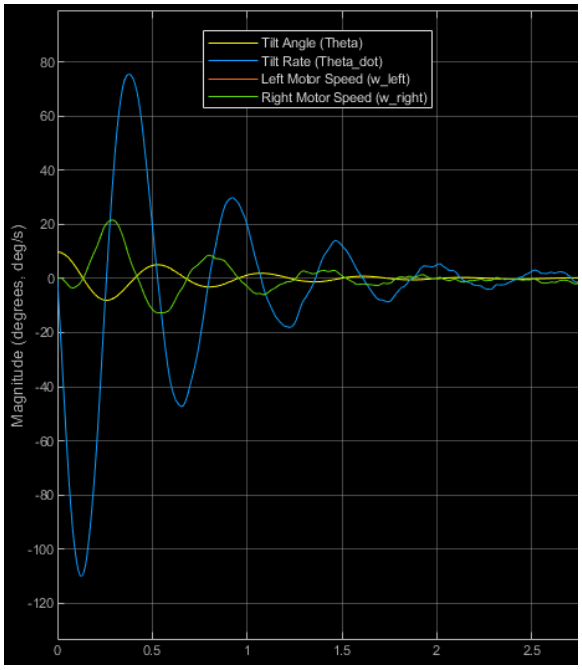
Test#3:


Fwd Veloc (x_dot)
Turn Veloc Rate (phi_dot)
Left Motor Current (I_left)
Right Motor Current (I_right)
Velocity (m/s), Current (A)

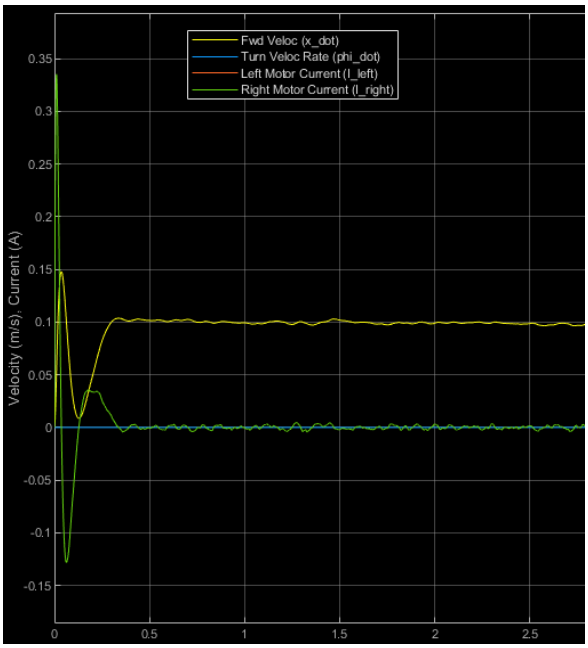

Tilt Angle (Theta)
Tilt Rate (Theta_dot)
Left Motor Speed (w_left)
Right Motor Speed (w_right)
Magnitude (degrees, deg/s)

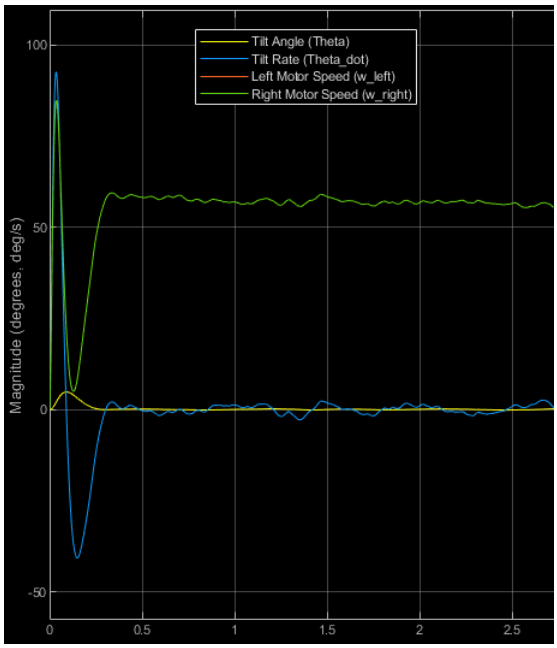
Test#4:


Fwd Veloc (x_dot)
Turn Veloc Rate (phi_dot)
Left Motor Current (I_left)
Right Motor Current (I_right)
Velocity (m/s), Current (A)

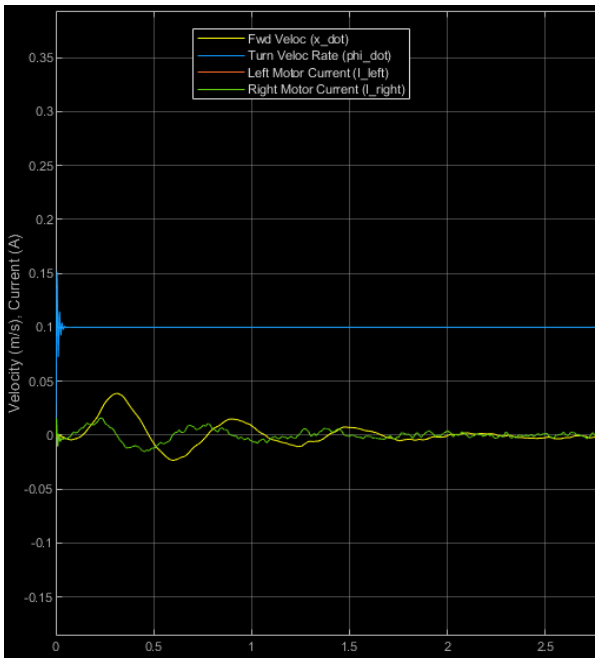

Tilt Angle (Theta)
Tilt Rate (Theta_dot)
Left Motor Speed (w_left)
Right Motor Speed (w_right)
Magnitude (degrees, deg/s)

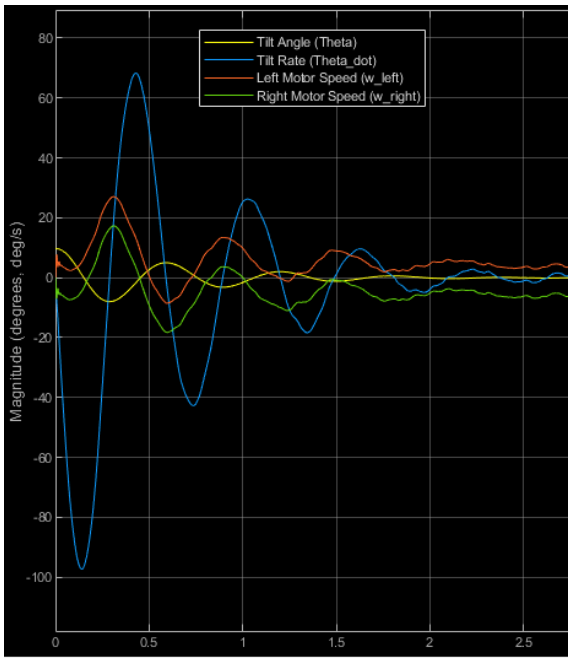
Test#5:


Fwd Veloc (x_dot)
Turn Veloc Rate (phi_dot)
Left Motor Current (I_left)
Right Motor Current (I_right)
Velocity (m/s), Current (A)

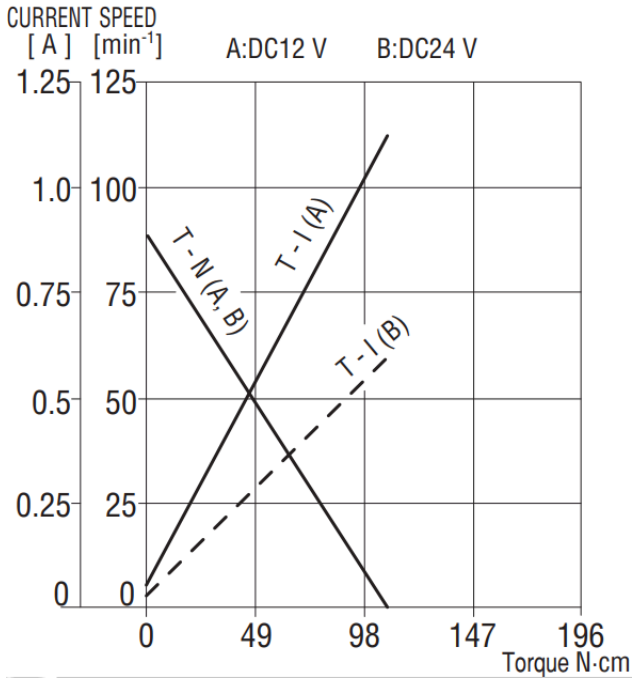

Tilt Angle (Theta)
Tilt Rate (Theta_dot)
Left Motor Speed (w_left)
Right Motor Speed (w_right)
Magnitude (degrees, deg/s)

Test#6:


Fwd Veloc (x_dot)
Turn Veloc Rate (phi_dot)
Left Motor Current (I_left)
Right Motor Current (I_right)
Velocity (m/s), Current (A)


Tilt Angle (Theta)
Tilt Rate (Theta_dot)
Left Motor Speed (w_left)
Right Motor Speed (w_right)
Magnitude (degrees, deg/s)

14. Brushed DC Motor Curve



Sources/References:

(1) Mass of arduino uno 25g
https://store-usa.arduino.cc/products/arduino-uno-rev3?srsltid=AfmBOor3l6FDzRnSYQguF87tadSfI2T8lzABoOi1PCjzghk9G-OPrYgz

(2) Motor characteristics:
https://www.nidec-components.com/e/catalog/dc-geared-motor/hg37.pdf

(3) Sensors and Actuators Clarence W. de Silva 2016

(4) https://howtomechatronics.com/tutorials/arduino/arduino-dc-motor-control-tutorial-l298n-pwm-h-bridge/

ECE 560 Winter 2025
Final Project Report
Scott Cann, Folarin Davies